

25ES611 - REAL TIME OPERATING SYSTEMS

FreeRTOS-Based Smart Attendance Monitoring System Using ESP32

PROJECT REPORT

Submitted by

MERAPALA HARIN – BL.EN.P2EBS25019

Guided: Dr. Surekha P

ELECTRICAL AND ELECTRONICS ENGINEERING



AMRITA SCHOOL OF ENGINEERING, BENGALURU

AMRITA VISHWA VIDYAPEETHAM

Bengaluru 560035

May -2026

ABSTRACT

The traditional attendance monitoring systems used in educational institutions mainly depend on manual attendance entry, biometric systems, or cloud-based applications that often suffer from limitations such as proxy attendance, high response delay, lack of real-time monitoring, and poor multitasking capability. Most existing attendance systems do not utilize Real-Time Operating System (RTOS) concepts for efficient task scheduling and concurrent operation handling in embedded environments.

To address these limitations, this project proposes a **FreeRTOS-Based Smart Attendance Monitoring System** using the ESP32 microcontroller, RC522 RFID reader, OLED display, Firebase cloud platform, and Serial Studio real-time visualization tool. The proposed system performs real-time RFID-based attendance monitoring using multiple FreeRTOS tasks including RFID scanning, attendance processing, Firebase communication, and timer-based scheduling operations.

The system continuously detects RFID cards through the RC522 RFID reader and processes attendance information using queue-based inter-task communication and mutex synchronization mechanisms. Attendance data including student name, UID, date, and time is uploaded to Firebase Realtime Database through WiFi communication for cloud-based attendance monitoring and storage.

The proposed architecture also includes OLED display notifications, buzzer indications, LED-based status alerts, session control using master RFID cards, duplicate attendance prevention, and unauthorized card detection mechanisms. Serial Studio is integrated with the system to visualize real-time task execution and RTOS scheduling behavior using hardware-generated telemetry graphs.

The implemented FreeRTOS-based architecture provides efficient multitasking capability, low-latency task scheduling, real-time cloud synchronization, and reliable embedded attendance monitoring functionality. The complete system was successfully implemented and tested on ESP32 hardware under multiple attendance conditions. Experimental results confirmed reliable RFID detection, stable FreeRTOS task execution, efficient queue communication, proper mutex synchronization, and real-time task visualization.

The developed system demonstrates an efficient and cost-effective real-time embedded attendance monitoring solution suitable for educational institutions and smart embedded monitoring applications.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION	6
1.1 Introduction.....	6
1.2 Problem Statement	7
1.3 Objectives	7
1.4 Motivation.....	8
1.5 Scope of the Project.....	9
1.6 Summary.....	10
CHAPTER 2 LITERATURE SURVEY	11
2.1 Introduction and Literature Review	11
2.2 Existing Attendance Monitoring Systems	12
2.3 RFID-Based Attendance Systems	13
2.4 RTOS-Based Embedded Monitoring Systems.....	14
2.5 Limitations of Existing Systems	15
2.6 Research Gap	17
2.7 Comparative Analysis	18
2.8 Summary.....	20
CHAPTER 3 PROPOSED SYSTEM	21
3.1 Introduction.....	21
3.2 System Overview	22
3.3 System Architecture	23
3.4 Block Diagram Description	25
3.5 FreeRTOS Task Architecture	26
3.6 RFID Attendance Monitoring Process	28
3.7 Queue-Based Inter-Task Communication	29
3.8 Mutex Synchronization Mechanism.....	31
3.9 Timer Task and Scheduling Operation.....	32
3.10 Firebase Cloud Integration.....	33
3.11 Serial Studio RTOS Visualization.....	35
3.12 Working Principle	36
3.13 Summary.....	38
CHAPTER 4 HARDWARE IMPLEMENTATION.....	38

4.1 Introduction.....	38
4.2 ESP32 Development Board	39
4.3 RC522 RFID Reader	41
4.4 OLED Display Module.....	42
4.5 Buzzer and LED Indicators.....	43
4.6 WiFi Communication Module	45
4.7 Power Supply Arrangement.....	46
4.8 Hardware Connections and Experimental Setup	47
4.9 Summary.....	49
CHAPTER 5 SOFTWARE IMPLEMENTATION	50
5.1 Introduction.....	50
5.2 Arduino IDE Development Environment	51
5.3 FreeRTOS Task Implementation	52
5.4 RFID Task Implementation	54
5.5 Attendance Processing Task	55
5.6 Firebase Communication Task	57
5.7 Queue Communication Logic	59
5.8 Mutex Synchronization Logic.....	61
5.9 Timer Task and RTOS Scheduling	63
5.10 Serial Studio Real-Time Monitoring	64
5.11 OLED Display Interface	66
5.12 Firebase Database Implementation	68
5.14 Summary.....	71
CHAPTER 6 RESULTS AND ANALYSIS	72
6.1 Introduction.....	72
6.2 RFID Detection Results.....	73
6.3 Attendance Monitoring Results	75
6.4 Firebase Cloud Upload Results.....	76
6.5 FreeRTOS Task Scheduling Analysis.....	78
6.6 Queue Communication Analysis	80
6.7 Mutex Synchronization Analysis	82

6.8 Serial Studio Graph Analysis.....	83
6.9 Hardware Output Analysis	86
6.10 Performance Evaluation.....	88
6.11 Summary	90
CHAPTER 7 CONCLUSION AND FUTURE SCOPE	91
7.1 Conclusion	91
7.2 Future Scope	92
7.3 Summary	94
CHAPTER 8 REFERENCES	95

FreeRTOS-Based Smart Attendance Monitoring System Using ESP32

CHAPTER 1 INTRODUCTION

1.1 Introduction

Attendance monitoring plays an important role in educational institutions, industries, and organizational environments for maintaining proper record management and performance evaluation. Traditional attendance systems generally depend on manual entry methods, paper-based registers, biometric systems, or standalone embedded systems. These methods often suffer from problems such as proxy attendance, data redundancy, slow processing, limited scalability, lack of real-time monitoring, and inefficient multitasking capability.

With the rapid advancement of embedded systems and Internet of Things (IoT) technologies, smart attendance monitoring systems have become an important research area in real-time embedded applications. RFID technology provides contactless identification capability, fast response time, and reliable attendance acquisition compared to conventional attendance methods. However, most existing RFID-based attendance systems operate using sequential programming techniques without proper implementation of Real-Time Operating System (RTOS) concepts.

Real-Time Operating Systems are designed for efficient multitasking, deterministic scheduling, resource sharing, and concurrent execution of multiple tasks in embedded systems. FreeRTOS is one of the most widely used lightweight open-source RTOS platforms suitable for real-time embedded applications. FreeRTOS enables implementation of multitasking features such as task scheduling, queue communication, mutex synchronization, timer management, and real-time monitoring.

The proposed project presents a **FreeRTOS-Based Smart Attendance Monitoring System** using ESP32 microcontroller, RC522 RFID reader, Firebase cloud integration, OLED display, buzzer indication, LED status monitoring, and Serial Studio visualization. The ESP32 microcontroller acts as the central processing unit and executes multiple FreeRTOS tasks concurrently for RFID detection, attendance validation, cloud synchronization, and timer-based scheduling operations.

The RFID RC522 reader is used for detecting RFID cards and extracting unique card identification numbers. Attendance information is processed using FreeRTOS queue communication and synchronization mechanisms. The attendance data is uploaded to Firebase Realtime Database using WiFi communication for cloud-based attendance monitoring and remote accessibility.

The system also integrates OLED display notifications, buzzer alerts, and LED indicators for real-time user interaction. Serial Studio is integrated with the system for visualization of RTOS

task execution behavior and real-time scheduling analysis using UART telemetry communication.

The proposed system demonstrates practical implementation of RTOS concepts in embedded attendance monitoring applications while ensuring reliable multitasking, real-time operation, cloud synchronization, and efficient task scheduling.

1.2 Problem Statement

Traditional attendance monitoring systems used in educational institutions mainly depend on manual attendance entry, paper-based registers, biometric systems, or standalone embedded applications. These systems often suffer from several limitations such as proxy attendance, human errors, slow attendance processing, lack of real-time monitoring, and inefficient data management.

Many existing attendance systems are designed using sequential programming techniques where all operations are executed one after another without proper multitasking support. As a result, simultaneous operations such as RFID scanning, attendance validation, cloud communication, and display handling cannot be managed efficiently. This leads to increased response delay, poor processor utilization, reduced system responsiveness, and unstable system performance.

Conventional embedded attendance systems also lack proper implementation of Real-Time Operating System concepts such as task scheduling, inter-task communication, synchronization mechanisms, and timer management. Without RTOS support, handling multiple concurrent operations becomes difficult in real-time embedded environments.

In addition, many existing systems do not provide real-time cloud synchronization and remote attendance monitoring capabilities. Some systems also fail to prevent duplicate attendance entries and unauthorized card access effectively. Lack of task visualization and scheduling analysis further limits understanding of real-time system behavior in embedded applications.

Therefore, there is a need for an efficient RTOS-based smart attendance monitoring system capable of performing real-time RFID attendance acquisition, concurrent task execution, cloud synchronization, task scheduling, and real-time monitoring using proper FreeRTOS mechanisms and embedded multitasking techniques.

1.3 Objectives

The main objective of the proposed project is to design and implement a **FreeRTOS-Based Smart Attendance Monitoring System** using ESP32 microcontroller and RFID technology for efficient real-time attendance management and RTOS task execution analysis.

The specific objectives of the proposed system are listed below:

- To develop a real-time RFID-based attendance monitoring system using ESP32 and RC522 RFID reader.
- To implement multiple concurrent tasks using FreeRTOS for RFID scanning, attendance processing, Firebase communication, and timer monitoring.
- To implement priority-based task scheduling for efficient multitasking and real-time system operation.
- To establish inter-task communication using FreeRTOS queue mechanisms for reliable data transfer between tasks.
- To implement mutex synchronization mechanisms for safe access to shared resources such as OLED display communication.
- To integrate Firebase Realtime Database for cloud-based attendance storage and remote monitoring.
- To implement attendance session control using master RFID cards.
- To prevent duplicate attendance entries and unauthorized RFID card access.
- To provide real-time user notifications using OLED display, buzzer alerts, and LED indicators.
- To visualize RTOS task scheduling and execution behavior using Serial Studio through UART telemetry communication.
- To analyze the performance of FreeRTOS multitasking mechanisms in embedded attendance monitoring applications.
- To demonstrate practical implementation of RTOS concepts such as task scheduling, queue communication, mutex synchronization, and software timers in real-time embedded systems.

1.4 Motivation

The rapid advancement of embedded systems, IoT technologies, and automation has increased the demand for efficient and intelligent attendance monitoring systems in educational institutions and organizations. Traditional attendance systems based on manual entry methods and paper-based registers are time consuming, error prone, and inefficient for real-time monitoring applications.

Many existing attendance systems also suffer from issues such as proxy attendance, delayed processing, lack of automation, poor multitasking capability, and inefficient data management. Sequential embedded systems without RTOS support cannot efficiently handle multiple concurrent operations such as RFID scanning, cloud synchronization, display handling, and notification management simultaneously.

The motivation behind this project is to develop a real-time embedded attendance monitoring system capable of implementing efficient multitasking and scheduling mechanisms using FreeRTOS. The integration of RTOS concepts such as task scheduling, queue communication, mutex synchronization, and timer management provides better processor utilization, reliable concurrent execution, and improved real-time responsiveness.

Another important motivation is to demonstrate practical implementation of FreeRTOS concepts in embedded systems using ESP32 hardware. Most academic projects focus mainly on hardware interfacing without proper RTOS implementation and task analysis. This project aims to bridge that gap by integrating real-time scheduling visualization and multitasking analysis using Serial Studio.

The integration of Firebase cloud synchronization and RFID-based attendance acquisition further improves the efficiency, automation capability, and remote accessibility of the attendance system. The proposed project also demonstrates practical application of RTOS concepts for real-time embedded monitoring systems suitable for educational and research applications.

1.5 Scope of the Project

The scope of the proposed **FreeRTOS-Based Smart Attendance Monitoring System** includes the design and implementation of a real-time RFID-based attendance management system using ESP32 microcontroller, FreeRTOS multitasking mechanisms, Firebase cloud integration, and Serial Studio task visualization.

The project focuses on implementing concurrent task execution using FreeRTOS for operations such as RFID scanning, attendance processing, Firebase communication, OLED display handling, and timer-based monitoring. The system demonstrates practical implementation of RTOS concepts including task scheduling, queue communication, mutex synchronization, and software timer management in embedded systems.

The RFID RC522 reader is used for contactless attendance acquisition and student identification. Attendance information including student name, UID, date, and time is processed and uploaded to Firebase Realtime Database through WiFi communication for cloud-based attendance storage and remote monitoring.

The project also includes implementation of attendance session control using master RFID cards, duplicate attendance prevention, unauthorized card detection, OLED display notifications, buzzer alerts, and LED status indication mechanisms for improving system reliability and user interaction.

Another important scope of the project is real-time visualization of RTOS task scheduling and execution behavior using Serial Studio. UART telemetry communication is used to transmit task execution traces for graphical scheduling analysis and multitasking visualization.

The proposed system is mainly intended for educational institutions, embedded systems laboratories, RTOS learning environments, and real-time embedded monitoring applications. The architecture can be further extended in the future using biometric authentication, mobile applications, AI-based attendance analysis, advanced cloud analytics, and large-scale institutional monitoring systems.

1.6 Summary

This chapter presented the introduction and background related to real-time attendance monitoring systems and RTOS-based embedded applications. Various traditional attendance systems, RFID-based monitoring approaches, and embedded multitasking techniques were studied to understand the current advancements and limitations in attendance management technologies.

The limitations of conventional attendance systems such as proxy attendance, manual processing, lack of multitasking capability, poor real-time responsiveness, inefficient task scheduling, and absence of proper RTOS implementation were discussed in detail. The need for a real-time embedded attendance system capable of performing concurrent task execution, cloud synchronization, and efficient multitasking using FreeRTOS concepts was identified.

The objectives, motivation, and scope of the proposed FreeRTOS-Based Smart Attendance Monitoring System were also presented. The proposed system integrates ESP32 microcontroller, RC522 RFID reader, Firebase cloud platform, OLED display, buzzer alerts, LED indicators, and Serial Studio visualization for efficient real-time attendance monitoring and RTOS task analysis.

The project demonstrates practical implementation of FreeRTOS concepts such as task scheduling, queue communication, mutex synchronization, software timers, and real-time task visualization in embedded systems. The next chapter presents the literature survey related to RFID-based attendance systems, RTOS-based embedded monitoring applications, existing attendance technologies, and research gaps addressed by the proposed system.

CHAPTER 2 LITERATURE SURVEY

2.1 Introduction and Literature Review

Attendance monitoring systems have become an important component in educational institutions, industries, and organizational environments for maintaining accurate attendance records and improving management efficiency. Various technologies such as manual attendance systems, biometric authentication, RFID-based identification, IoT-based monitoring, and embedded automation systems have been developed to improve attendance management and reduce manual effort.

Traditional attendance systems mainly depend on paper-based registers and manual attendance entry methods. Although these systems are simple to implement, they suffer from several limitations including proxy attendance, human errors, time consumption, data redundancy, and inefficient record management. The absence of automation and real-time monitoring further reduces system reliability and scalability.

To overcome these limitations, RFID-based attendance systems were introduced for contactless identification and automatic attendance acquisition. RFID technology enables fast attendance detection, reduced human intervention, and efficient student identification using RFID cards and readers. Several research works implemented RFID-based attendance systems using microcontrollers and IoT platforms for automated attendance monitoring. However, many of these systems mainly focus on attendance acquisition without implementing proper multitasking mechanisms and RTOS-based scheduling techniques.

Recent advancements in embedded systems and Internet of Things technologies enabled development of cloud-based attendance monitoring systems using WiFi communication and remote databases. Firebase cloud integration, mobile applications, and web dashboards are commonly used in modern attendance systems for remote attendance access and cloud storage. Although these systems improve accessibility and automation, many of them still rely on sequential programming approaches without efficient real-time task scheduling and concurrent task execution.

Real-Time Operating Systems have gained significant importance in embedded applications requiring deterministic execution, multitasking capability, resource sharing, and efficient scheduling mechanisms. FreeRTOS is one of the most widely used lightweight RTOS platforms suitable for embedded systems and IoT applications. RTOS-based architectures provide efficient task scheduling, queue communication, synchronization mechanisms, software timers, and improved processor utilization compared to conventional sequential systems.

Several research works have implemented RTOS concepts in embedded automation systems such as industrial monitoring, smart home applications, IoT devices, and sensor-based monitoring systems. However, limited research has been carried out on integrating FreeRTOS multitasking concepts with RFID-based attendance monitoring systems along with real-time task visualization and cloud synchronization.

The proposed FreeRTOS-Based Smart Attendance Monitoring System addresses these limitations by integrating RFID attendance acquisition, FreeRTOS multitasking, queue communication, mutex synchronization, Firebase cloud integration, and Serial Studio-based RTOS task visualization within a single embedded architecture.

2.2 Existing Attendance Monitoring Systems

Existing attendance monitoring systems mainly focus on recording and maintaining attendance information using manual methods, biometric authentication, RFID technology, IoT platforms, and cloud-based monitoring systems. Various technologies have been developed over the years to improve attendance management efficiency and reduce manual effort in educational institutions and organizations.

Traditional attendance systems generally use paper-based registers and manual attendance entry procedures. In these systems, attendance is recorded manually by teachers or administrators during class sessions. Although manual systems are simple and cost effective, they suffer several limitations such as human errors, time consumption, proxy attendance, data redundancy, and difficulty in maintaining large attendance records.

To overcome these limitations, biometric attendance systems using fingerprint sensors and facial recognition technologies were introduced. These systems improved attendance authenticity and reduced proxy attendance problems. However, biometric systems often require expensive hardware, complex database management, higher processing requirements, and increased maintenance cost. In some cases, biometric systems may also experience slow response time and environmental sensitivity issues.

RFID-based attendance monitoring systems were later developed to provide contactless attendance acquisition and automatic student identification. In these systems, RFID readers detect RFID cards carried by students and automatically record attendance information. RFID systems provide faster operation, reduced human interaction, low-cost implementation, and efficient attendance acquisition compared to conventional attendance methods.

Several IoT-based attendance systems integrate RFID technology with WiFi communication, cloud databases, and mobile applications for remote attendance monitoring and cloud synchronization. Firebase databases, mobile dashboards, and web-based interfaces are commonly

used for storing and visualizing attendance information remotely. These systems improve accessibility and enable cloud-based attendance management.

Although existing RFID and IoT-based attendance systems improve automation and attendance acquisition efficiency, many of them still suffer from several limitations. Most existing systems are developed using sequential programming approaches without proper implementation of Real-Time Operating System concepts. As a result, simultaneous operations such as RFID scanning, database communication, display handling, and notification management cannot be handled efficiently.

Many existing systems also lack proper task scheduling mechanisms, inter-task communication methods, synchronization techniques, and real-time monitoring capability. In addition, several systems do not support RTOS-based task visualization and scheduling analysis, limiting their applicability in real-time embedded system research and multitasking applications.

These limitations highlight the need for an RTOS-based smart attendance monitoring system capable of performing efficient multitasking, concurrent task execution, cloud synchronization, and real-time scheduling visualization using embedded RTOS mechanisms.

2.3 RFID-Based Attendance Systems

RFID-based attendance systems are widely used in educational institutions and organizational environments for automatic attendance monitoring and contactless identification. RFID technology provides efficient wireless communication between RFID tags and RFID readers, enabling fast and reliable attendance acquisition without manual intervention.

An RFID system mainly consists of RFID tags, RFID readers, microcontrollers, and database management systems. Each user is assigned a unique RFID card containing a specific identification number known as UID. When the RFID card is placed near the RFID reader, the reader detects the UID and transmits the information to the processing unit for attendance verification and storage.

Several research works have implemented RFID-based attendance systems using microcontrollers such as Arduino, ESP8266, NodeMCU, Raspberry Pi, and ESP32 platforms. These systems automatically record attendance information and reduce the limitations associated with traditional manual attendance systems.

RFID-based attendance systems provide several advantages such as fast attendance acquisition, reduced human errors, low-cost implementation, contactless operation, easy scalability, and efficient student identification. Many systems also integrate WiFi communication and cloud databases for online attendance storage and remote accessibility.

Some advanced RFID attendance systems integrate IoT platforms, mobile applications, and web dashboards for real-time attendance monitoring and analytics. Firebase Realtime Database and cloud-based storage systems are commonly used for remote attendance management and synchronization.

Although RFID-based attendance systems improve automation and attendance acquisition efficiency, several limitations still exist in many implementations. Most RFID attendance systems are developed using sequential embedded programming methods without proper multitasking support. As a result, concurrent operations such as RFID scanning, attendance processing, cloud synchronization, display handling, and notification management cannot be executed efficiently.

Many existing RFID attendance systems also lack implementation of RTOS concepts such as task scheduling, queue communication, synchronization mechanisms, software timers, and concurrent task management. Without RTOS support, processor utilization becomes inefficient and real-time responsiveness decreases during multiple simultaneous operations.

In addition, many existing systems do not provide real-time visualization of embedded task execution and scheduling behavior. The absence of RTOS task monitoring and scheduling analysis limits understanding of multitasking concepts in embedded systems.

The proposed FreeRTOS-Based Smart Attendance Monitoring System addresses these limitations by integrating RFID attendance acquisition with FreeRTOS multitasking mechanisms, queue communication, mutex synchronization, Firebase cloud integration, and Serial Studio-based RTOS task visualization for efficient real-time embedded operation.

2.4 RTOS-Based Embedded Monitoring Systems

Real-Time Operating System (RTOS)-based embedded monitoring systems have gained significant importance in modern embedded applications due to their ability to perform multitasking, concurrent task execution, deterministic scheduling, and efficient resource management. RTOS platforms are widely used in applications requiring real-time responsiveness, reliable scheduling, and simultaneous handling of multiple operations.

Traditional embedded systems developed using sequential programming approaches execute operations one after another without proper multitasking support. Such systems often suffer from increased response delay, inefficient processor utilization, poor scheduling efficiency, and limited capability to handle multiple concurrent operations. These limitations become critical in real-time embedded applications where timing accuracy and multitasking are essential.

To overcome these limitations, RTOS-based architectures were introduced for real-time embedded monitoring and automation systems. RTOS provides important features such as task scheduling, inter-task communication, synchronization mechanisms, timer management, priority handling, and efficient processor sharing. These features improve real-time responsiveness and overall system performance.

FreeRTOS is one of the most widely used lightweight open-source RTOS kernels designed for embedded systems and IoT applications. FreeRTOS supports preemptive scheduling, multitasking, queues, semaphores, mutexes, software timers, and concurrent task management suitable for resource-constrained microcontroller platforms such as ESP32.

Several research works have implemented RTOS concepts in embedded monitoring applications such as industrial automation, environmental monitoring, smart home systems, healthcare monitoring, wireless sensor networks, and IoT-based control systems. These systems utilize FreeRTOS mechanisms for handling multiple sensor inputs, communication modules, cloud synchronization, and real-time control operations concurrently.

RTOS-based embedded architectures provide several advantages including improved multitasking capability, deterministic execution, better processor utilization, reduced latency, efficient scheduling, and reliable concurrent task execution. Queue communication mechanisms enable safe data transfer between tasks, while mutex synchronization protects shared resources from simultaneous access conflicts.

Although many embedded monitoring systems implement RTOS concepts for multitasking applications, limited research has focused on integrating RTOS-based scheduling and visualization mechanisms in RFID attendance monitoring systems. Most existing systems lack real-time task monitoring, scheduling analysis, and practical demonstration of RTOS concepts using embedded visualization tools.

The proposed FreeRTOS-Based Smart Attendance Monitoring System addresses these limitations by implementing multiple FreeRTOS tasks for RFID scanning, attendance processing, Firebase communication, and timer monitoring. The system also integrates Serial Studio for real-time visualization of task scheduling and multitasking behavior using UART telemetry communication.

2.5 Limitations of Existing Systems

The detailed study of existing attendance monitoring systems, RFID-based attendance architectures, and embedded monitoring applications revealed several limitations in current approaches. Although many systems provide automatic attendance acquisition and cloud-based

monitoring capability, most existing systems still suffer from problems related to multitasking efficiency, real-time responsiveness, scheduling mechanisms, and embedded task management.

Traditional attendance systems mainly depend on manual entry methods and sequential processing approaches. These systems are time consuming, prone to human errors, and inefficient for handling large attendance databases. Proxy attendance and duplicate attendance issues are also common in manual attendance systems.

Many biometric attendance systems improve authentication accuracy but require expensive hardware, complex database management, and high processing requirements. In addition, biometric systems may experience slow response time, environmental sensitivity issues, and higher maintenance cost.

Existing RFID-based attendance systems improve attendance acquisition speed and automation capability; however, most of these systems are implemented using sequential embedded programming without proper RTOS support. As a result, simultaneous operations such as RFID scanning, attendance processing, cloud synchronization, OLED display handling, and notification generation cannot be managed efficiently.

Several IoT-based attendance systems integrate cloud databases and WiFi communication for remote monitoring, but many systems suffer from communication delays, inefficient processor utilization, and unstable performance during concurrent operations. Lack of deterministic task scheduling further reduces system responsiveness in real-time embedded environments.

Most existing systems also lack implementation of important RTOS mechanisms such as:

- Priority-based task scheduling
- Inter-task queue communication
- Mutex synchronization
- Software timer management
- Concurrent task execution
- Real-time scheduling analysis

Without proper synchronization mechanisms, shared resources such as display modules and communication interfaces may experience simultaneous access conflicts and unstable operation.

Another important limitation is the absence of real-time RTOS task visualization and scheduling monitoring in many embedded attendance systems. Existing systems generally focus only on attendance functionality without demonstrating task scheduling behavior, processor activity, and multitasking execution flow.

In addition, several systems do not support efficient duplicate attendance prevention, session-based attendance control, unauthorized RFID card detection, and embedded scheduling analysis suitable for RTOS-based educational and research applications.

These limitations highlight the need for a FreeRTOS-based smart attendance monitoring system capable of efficient multitasking, reliable scheduling, queue communication, synchronization management, cloud integration, and real-time task visualization using embedded RTOS mechanisms.

2.6 Research Gap

The detailed analysis of existing attendance monitoring systems, RFID-based attendance architectures, IoT-enabled monitoring systems, and RTOS-based embedded applications revealed several research gaps and limitations in current approaches.

Most traditional attendance systems mainly focus on attendance recording and cloud synchronization without implementing efficient real-time multitasking and scheduling mechanisms. Many existing systems are developed using sequential programming approaches where operations are executed one after another without proper concurrent task management. This reduces processor utilization efficiency and increases response delay during simultaneous operations.

Several RFID-based attendance systems provide automatic attendance acquisition and remote monitoring capabilities; however, many of these systems lack proper implementation of Real-Time Operating System concepts such as:

- Priority-based task scheduling
- Queue-based inter-task communication
- Mutex synchronization
- Software timer management
- Concurrent task execution
- Deterministic scheduling behavior

Most existing systems also do not provide proper resource synchronization mechanisms for shared peripherals such as displays and communication interfaces. The absence of synchronization methods may lead to unstable operation and resource conflicts during concurrent task execution.

Another important research gap identified is the lack of real-time RTOS task visualization and scheduling analysis in embedded attendance systems. Existing RFID attendance systems mainly

focus on attendance functionality and cloud communication without demonstrating practical scheduling behavior, multitasking execution flow, and processor activity analysis.

Many current attendance systems also lack implementation of:

- Session-based attendance management
- Duplicate attendance prevention
- Unauthorized card detection
- Embedded scheduling analysis
- Real-time task monitoring
- Hardware-level RTOS execution visualization

Although several RTOS-based embedded monitoring systems exist, limited research has been carried out on integrating FreeRTOS multitasking concepts with RFID attendance monitoring, cloud synchronization, and real-time task visualization within a single embedded architecture.

To address these research gaps, the proposed FreeRTOS-Based Smart Attendance Monitoring System introduces an integrated RTOS-based embedded architecture using ESP32, RC522 RFID reader, Firebase cloud integration, FreeRTOS multitasking mechanisms, queue communication, mutex synchronization, software timers, and Serial Studio-based task visualization for efficient real-time attendance monitoring and embedded scheduling analysis.

2.7 Comparative Analysis

A comparative study was carried out between traditional attendance systems, RFID-based attendance systems, IoT-enabled attendance systems, and the proposed FreeRTOS-Based Smart Attendance Monitoring System. The analysis mainly focuses on attendance automation, multitasking capability, RTOS implementation, cloud synchronization, task scheduling, and real-time monitoring features.

Traditional manual attendance systems mainly rely on paper-based registers and manual entry methods. These systems are simple to implement but suffer from several limitations such as proxy attendance, human errors, increased processing time, and poor data management capability. Manual systems also do not support automation, cloud synchronization, or real-time monitoring.

Biometric attendance systems improve authentication accuracy using fingerprint or facial recognition technologies. Although these systems reduce proxy attendance problems, they require expensive hardware, higher maintenance cost, complex database management, and increased processing requirements. Most biometric systems also lack proper RTOS-based multitasking support.

RFID-based attendance systems provide contactless attendance acquisition and automatic student identification using RFID cards and readers. These systems improve attendance acquisition speed and reduce manual effort. However, many RFID attendance systems are implemented using sequential programming approaches without efficient task scheduling and synchronization mechanisms.

IoT-based attendance systems integrate cloud databases, WiFi communication, and mobile applications for remote attendance monitoring. These systems improve accessibility and cloud synchronization capability. However, several systems still lack proper multitasking support, deterministic scheduling, queue communication, and RTOS visualization mechanisms.

The proposed FreeRTOS-Based Smart Attendance Monitoring System overcomes these limitations by integrating FreeRTOS multitasking mechanisms, queue communication, mutex synchronization, software timers, Firebase cloud integration, and Serial Studio-based RTOS task visualization within a unified embedded architecture.

The comparative analysis of different attendance systems is presented below.

Features	Manual System	Biometric System	RFID System	Proposed RTOS-Based System
Automatic Attendance	No	Yes	Yes	Yes
Contactless Operation	No	Partial	Yes	Yes
Cloud Integration	No	Limited	Limited	Yes
RTOS Support	No	No	No	Yes
Multitasking Capability	No	Limited	Limited	Yes
Queue Communication	No	No	No	Yes
Mutex Synchronization	No	No	No	Yes
Real-Time Scheduling	No	No	No	Yes
Task Visualization	No	No	No	Yes
Duplicate Detection	Limited	Yes	Limited	Yes
Unauthorized Access Detection	No	Yes	Limited	Yes
Cost Efficiency	High	Low	Medium	High

Real-Time Monitoring	No	Partial	Partial	Yes
----------------------	----	---------	---------	-----

From the comparative analysis, it is observed that the proposed system provides significant improvements in multitasking capability, RTOS implementation, real-time monitoring, scheduling analysis, synchronization mechanisms, and cloud-based attendance management compared to existing attendance systems.

2.8 Summary

This chapter presented a detailed literature survey related to attendance monitoring systems, RFID-based attendance architectures, IoT-enabled monitoring systems, and RTOS-based embedded applications. Various traditional attendance systems, biometric systems, RFID attendance systems, and cloud-based monitoring approaches were studied to understand the current developments and limitations in attendance management technologies.

The study showed that traditional attendance systems suffer from several limitations such as proxy attendance, manual processing, human errors, poor scalability, and lack of automation. RFID-based attendance systems improved attendance acquisition efficiency and reduced manual effort; however, many existing systems still rely on sequential programming approaches without proper multitasking capability and RTOS implementation.

The literature survey also highlighted the importance of Real-Time Operating Systems in embedded applications requiring concurrent task execution, deterministic scheduling, resource synchronization, and efficient processor utilization. FreeRTOS provides important features such as task scheduling, queue communication, mutex synchronization, software timers, and multitasking support suitable for real-time embedded systems.

The limitations of existing attendance systems and embedded monitoring architectures were analyzed in detail. Important research gaps identified include lack of RTOS-based attendance monitoring, absence of queue communication and synchronization mechanisms, inefficient concurrent task handling, and lack of real-time scheduling visualization.

A comparative analysis between traditional attendance systems and the proposed FreeRTOS-Based Smart Attendance Monitoring System was also presented. The analysis showed that the proposed system provides significant improvements in multitasking capability, RTOS implementation, cloud synchronization, scheduling analysis, and real-time monitoring compared to existing approaches.

The next chapter presents the proposed system architecture, FreeRTOS task organization, system workflow, hardware-software integration, and overall methodology of the proposed smart attendance monitoring system.

CHAPTER 3 PROPOSED SYSTEM

3.1 Introduction

This chapter presents the proposed architecture and methodology of the FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, RFID technology, Firebase cloud integration, and real-time RTOS task visualization mechanisms. The proposed system is designed to provide efficient attendance monitoring, concurrent task execution, cloud synchronization, and embedded scheduling analysis using FreeRTOS multitasking concepts.

The system integrates both hardware and software components for performing real-time attendance acquisition and monitoring operations. The ESP32 microcontroller acts as the central processing unit responsible for RFID scanning, attendance validation, cloud communication, OLED display handling, and RTOS task scheduling.

The proposed architecture utilizes the RC522 RFID reader for contactless attendance acquisition and student identification. Multiple FreeRTOS tasks are implemented for handling RFID detection, attendance processing, Firebase communication, and timer-based monitoring concurrently. Queue communication mechanisms and mutex synchronization techniques are used to ensure safe and efficient inter-task communication and resource management.

The system also integrates Firebase Realtime Database for cloud-based attendance storage and remote monitoring. Attendance details including student name, RFID UID, date, and time are uploaded to the cloud database through WiFi communication. OLED display notifications, buzzer alerts, and LED indicators provide real-time user interaction and attendance status indication.

An important feature of the proposed system is the implementation of RTOS task visualization using Serial Studio. UART telemetry communication is used to transmit task execution traces for graphical scheduling analysis and real-time multitasking visualization.

This chapter explains the proposed system architecture, hardware-software integration, FreeRTOS task organization, queue communication mechanisms, synchronization techniques, timer operations, and overall working methodology of the proposed smart attendance monitoring system.

3.2 System Overview

The proposed FreeRTOS-Based Smart Attendance Monitoring System is designed to perform real-time RFID-based attendance acquisition, multitasking execution, cloud synchronization, and RTOS scheduling visualization using ESP32 microcontroller and FreeRTOS.

The system integrates hardware modules, software components, IoT communication, and RTOS mechanisms to achieve efficient attendance monitoring and embedded multitasking operation. The major hardware components used in the system include ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, and WiFi communication interface.

The ESP32 microcontroller acts as the central processing unit of the system and executes multiple FreeRTOS tasks concurrently. The RC522 RFID reader continuously scans RFID cards and detects student RFID UIDs for attendance acquisition. The detected RFID information is processed through separate FreeRTOS tasks using queue-based inter-task communication mechanisms.

The proposed system implements multiple FreeRTOS tasks for different operations including:

- RFID scanning task
- Attendance processing task
- Firebase communication task
- Timer monitoring task

The RFID Task continuously monitors RFID card detection and sends UID information to the Processing Task through FreeRTOS queues. The Processing Task validates the RFID card, checks attendance session status, prevents duplicate attendance entries, and identifies unauthorized RFID cards.

Attendance session control is implemented using a master RFID card. When the master card is scanned, the attendance session is activated or deactivated. Student attendance is recorded only during active attendance sessions.

After successful attendance verification, attendance details such as student name, UID, date, and time are transmitted to the Firebase Task through queue communication. The Firebase Task uploads attendance information to Firebase Realtime Database using WiFi communication for cloud-based attendance monitoring and storage.

The OLED display provides real-time notifications including attendance confirmation, duplicate attendance warning, session status, and unauthorized card alerts. The buzzer and LED indicators provide audio-visual feedback during attendance operations.

Mutex synchronization mechanisms are implemented for safe access to shared resources such as OLED display communication. FreeRTOS software timers are used for periodic monitoring and RTOS scheduling analysis.

The system also integrates Serial Studio for real-time RTOS task visualization. Task execution traces generated through UART telemetry communication are displayed graphically for analyzing multitasking behavior and scheduling flow.

The proposed architecture ensures efficient multitasking, reliable attendance acquisition, real-time cloud synchronization, concurrent task execution, and practical implementation of RTOS concepts in embedded systems.

3.3 System Architecture

The system architecture of the proposed FreeRTOS-Based Smart Attendance Monitoring System consists of hardware modules, software components, cloud services, and RTOS task management mechanisms integrated to perform real-time attendance monitoring efficiently.

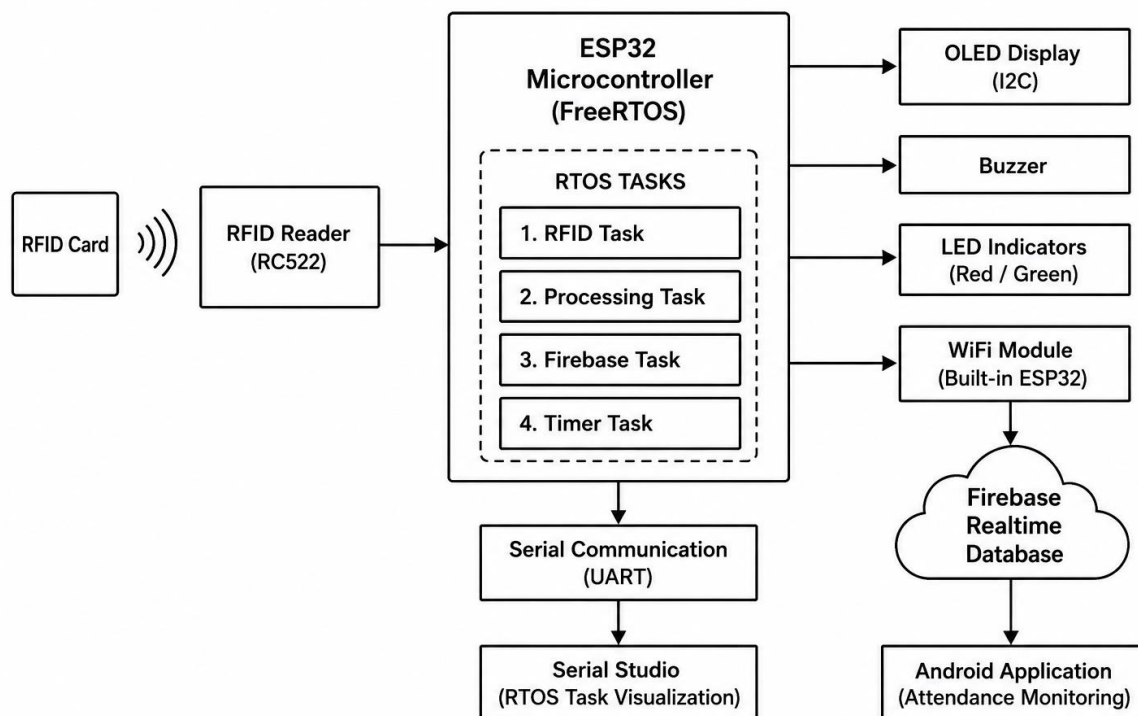


Figure 3.1: Block Diagram of Proposed FreeRTOS-Based Smart Attendance Monitoring System

The ESP32 microcontroller acts as the central processing unit of the system. It interfaces with the RC522 RFID reader for RFID card detection, OLED display module for user notifications, buzzer and LED indicators for audio-visual alerts, Firebase cloud platform for attendance storage, and Serial Studio for RTOS task visualization.

The overall system operation begins when an RFID card is scanned using the RC522 RFID reader. The RFID Task running under FreeRTOS continuously monitors RFID card detection and extracts the UID information from the detected RFID card. The acquired RFID UID is then transferred to the Processing Task using FreeRTOS to queue communication mechanisms.

The Processing Task validates the RFID information and determines whether the scanned card belongs to a master card or a student card. The master RFID card is used for starting and stopping the attendance session. Student RFID cards are used for attendance marking during active attendance sessions.

The Processing Task also performs duplicate attendance verification and unauthorized card detection. If the attendance is valid, attendance details such as student name, UID, date, and time are prepared and transmitted to the Firebase Task through another FreeRTOS queue.

The Firebase Task establishes communication with Firebase Realtime Database through the WiFi interface of ESP32 and uploads attendance information to the cloud database. The cloud-based database enables remote attendance monitoring and attendance record management.

The OLED display module provides real-time system notifications such as attendance confirmation, session status indication, duplicate attendance warning, and invalid card detection. The buzzer and LED indicators provide immediate audio and visual feedback during attendance operations.

The proposed architecture utilizes FreeRTOS for efficient multitasking and concurrent task execution. Separate FreeRTOS tasks are implemented for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring. Queue communication mechanisms enable safe inter-task data transfer, while mutex synchronization mechanisms ensure protected access to shared resources such as OLED display communication.

A FreeRTOS software timer is used for periodic system monitoring and RTOS task trace generation. Serial Studio is integrated with the system to visualize real-time task execution behavior and scheduling flow using UART telemetry communication.

The overall system architecture ensures reliable attendance monitoring, efficient multitasking, real-time cloud synchronization, proper RTOS scheduling, and concurrent execution of embedded tasks within the proposed attendance monitoring system.

3.4 Block Diagram Description

The block diagram of the proposed FreeRTOS-Based Smart Attendance Monitoring System represents the overall hardware and software integration used for real-time attendance monitoring and RTOS task execution. The architecture consists of ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, Firebase cloud platform, and Serial Studio visualization interface.

The ESP32 microcontroller acts as the central controller of the entire system. It manages RFID detection, attendance processing, cloud communication, task scheduling, and real-time monitoring operations using FreeRTOS multitasking mechanisms.

The RC522 RFID reader is connected to the ESP32 through SPI communication interface. The RFID reader continuously scans RFID cards and transmits RFID UID information to the ESP32 for attendance verification and processing. Each RFID card contains a unique identification number used for student authentication.

The OLED display module is connected to the ESP32 through I2C communication and is used for displaying real-time attendance notifications, session status messages, duplicate attendance warnings, and invalid card alerts. The buzzer and LED indicators provide audio and visual feedback during attendance operations.

The system utilizes WiFi communication capability available in ESP32 for connecting with Firebase Realtime Database. Attendance information including student name, RFID UID, date, and time is uploaded to the cloud database for remote attendance monitoring and storage.

The proposed system implements multiple FreeRTOS tasks for handling different operations concurrently. Separate tasks are created for RFID scanning, attendance processing, Firebase communication, and timer monitoring. Queue communication mechanisms are used for safe transfer of attendance data between tasks, while mutex synchronization mechanisms protect shared resources from simultaneous access conflicts.

The timer task periodically generates RTOS scheduling traces and monitoring information. These task traces are transmitted through UART serial communication to Serial Studio for graphical visualization of task scheduling behavior and multitasking execution flow.

The block diagram demonstrates the interaction between hardware modules, cloud services, and RTOS mechanisms implemented within the proposed system. It also illustrates the complete data flow starting from RFID card detection to cloud synchronization and task visualization.

The architecture ensures efficient multitasking, reliable attendance acquisition, proper task synchronization, cloud connectivity, and real-time RTOS scheduling analysis within the embedded attendance monitoring system.

3.5 FreeRTOS Task Architecture

The proposed FreeRTOS-Based Smart Attendance Monitoring System utilizes a multitasking architecture implemented using FreeRTOS for efficient concurrent execution of multiple embedded operations. The FreeRTOS task architecture enables independent execution of RFID scanning, attendance processing, cloud synchronization, and timer-based monitoring within the system.

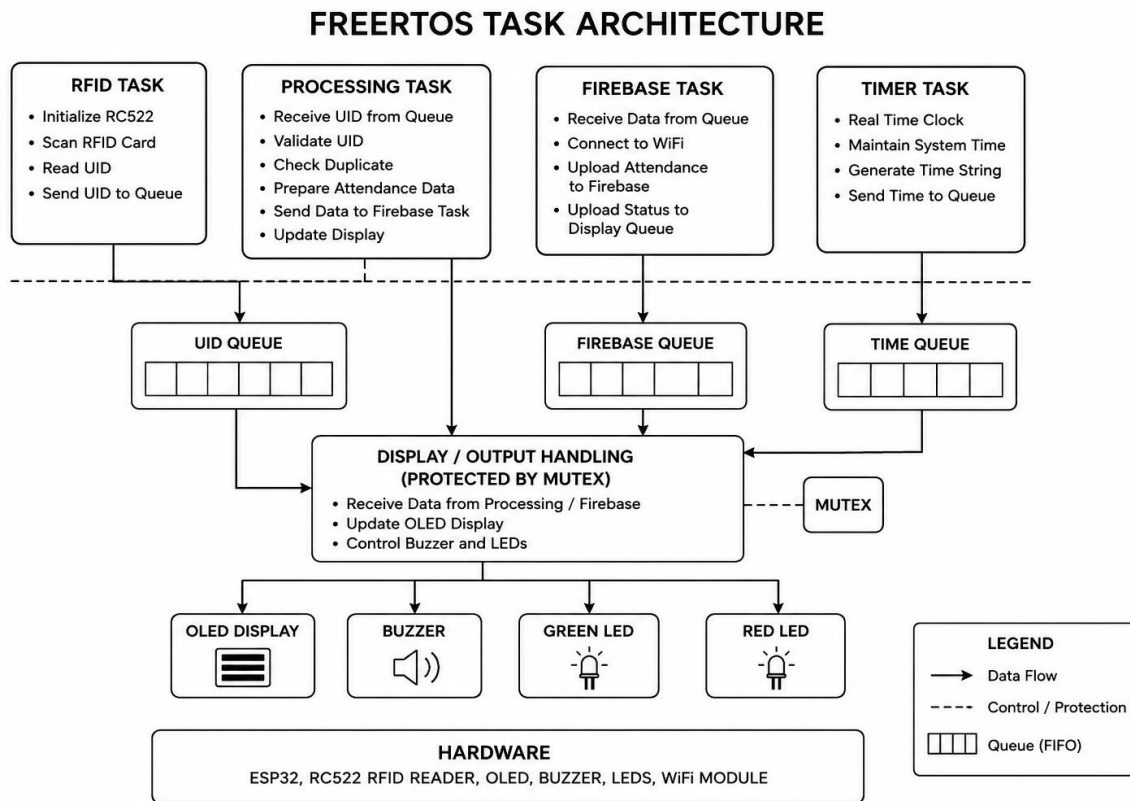


Figure 3.2: FreeRTOS Task Architecture and Queue Communication Flow

FreeRTOS provides a lightweight real-time operating system environment suitable for embedded systems requiring multitasking capability, deterministic scheduling, inter-task communication, and resource synchronization. In the proposed system, multiple FreeRTOS tasks are created with different priorities based on the importance and execution requirements of each operation.

The architecture consists of the following major FreeRTOS tasks:

- RFID Task
- Processing Task
- Firebase Task
- Timer Task

The RFID Task continuously monitors RFID card detection using the RC522 RFID reader. This task operates with higher priority because RFID scanning requires immediate response for real-time attendance acquisition. Whenever an RFID card is detected, the RFID UID information is extracted and transmitted to the Processing Task through FreeRTOS queue communication.

The Processing Task performs attendance validation, master card verification, duplicate attendance detection, and unauthorized card checking. This task receives RFID information from the RFID queue and processes attendance data based on the current attendance session status.

The Firebase Task handles cloud synchronization and attendance data uploading operations. Attendance information generated by the Processing Task is transferred to the Firebase Task through another queue. The Firebase Task establishes communication with Firebase Realtime Database through WiFi connectivity and uploads attendance details to the cloud server.

The Timer Task is responsible for periodic monitoring and RTOS task trace generation. This task executes periodically using FreeRTOS software timers and transmits task scheduling information to Serial Studio for real-time scheduling analysis and visualization.

Queue communication mechanisms are implemented for safe and reliable data transfer between concurrent tasks. Mutex synchronization techniques are used to protect shared resources such as OLED display communication from simultaneous task access conflicts.

The FreeRTOS scheduler manages execution of all tasks based on priority levels and task readiness. Higher-priority tasks are executed before lower-priority tasks, ensuring efficient real-time responsiveness and proper processor utilization.

The FreeRTOS task architecture implemented in the proposed system provides several advantages including:

- Efficient multitasking capability
- Concurrent task execution
- Improved processor utilization
- Faster real-time response
- Reliable task synchronization
- Safe inter-task communication
- Better scheduling efficiency

- Stable embedded system operation

The implemented task architecture demonstrates practical application of RTOS concepts in real-time embedded attendance monitoring systems using FreeRTOS and ESP32 microcontroller.

3.6 RFID Attendance Monitoring Process

The RFID attendance monitoring process is the primary operational mechanism of the proposed FreeRTOS-Based Smart Attendance Monitoring System. The process involves RFID card detection, attendance verification, task communication, cloud synchronization, and user notification using FreeRTOS multitasking mechanisms.

The attendance monitoring operation begins when an RFID card is placed near the RC522 RFID reader. The RFID reader continuously scans for RFID tags and detects the RFID UID associated with the card. The RFID Task running under FreeRTOS performs continuous RFID monitoring and extracts the RFID UID information whenever a valid card is detected.

After RFID detection, the RFID UID is transferred to the Processing Task using FreeRTOS queue communication. Queue mechanisms ensure safe and reliable transfer of RFID data between concurrent tasks without data corruption or synchronization issues.

The Processing Task receives the RFID UID and verifies whether the scanned card belongs to a master RFID card or a student RFID card. The master card is used for controlling attendance sessions. When the master card is scanned, the attendance session is activated or terminated depending on the current system state.

During an active attendance session, student RFID cards are accepted for attendance marking. The Processing Task checks whether the scanned student RFID card is authorized and verifies whether attendance has already been marked previously. If duplicate attendance is detected, the system generates duplicate attendance warnings through OLED display notifications and buzzer indications.

For valid student attendance, the system records attendance information including student name, RFID UID, date, and time. The attendance data is then transferred to the Firebase Task using another FreeRTOS queue for cloud synchronization.

The Firebase Task establishes communication with Firebase Realtime Database through WiFi connectivity and uploads attendance information to the cloud server. This enables real-time attendance storage and remote attendance monitoring.

The OLED display module provides real-time status notifications such as attendance confirmation, session activation, duplicate attendance alerts, and unauthorized card warnings.

The buzzer and LED indicators provide immediate audio-visual feedback during attendance operations.

The entire attendance monitoring process operates concurrently using FreeRTOS multitasking architecture. Queue communication, mutex synchronization, and priority-based scheduling ensure efficient and reliable operation of all attendance-related tasks within the system.

The RFID attendance monitoring process implemented in the proposed system provides several advantages including:

- Fast RFID-based attendance acquisition
- Contactless attendance monitoring
- Real-time attendance processing
- Duplicate attendance prevention
- Unauthorized card detection
- Cloud-based attendance storage
- Concurrent multitasking operation
- Efficient real-time system performance

The implemented RFID attendance monitoring process demonstrates reliable integration of RFID technology, FreeRTOS multitasking, and cloud synchronization within a real-time embedded attendance management system.

3.7 Queue-Based Inter-Task Communication

Queue-based inter-task communication is an important feature implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for safe and reliable data transfer between multiple concurrent tasks. FreeRTOS queues provide organized communication mechanisms that allow tasks to exchange information without direct dependency on each other.

In the proposed system, multiple FreeRTOS tasks operate concurrently for RFID scanning, attendance processing, Firebase communication, and timer monitoring. Since these tasks execute independently, an efficient communication mechanism is required for transferring RFID information and attendance data safely between tasks. FreeRTOS queue mechanisms are implemented to achieve this functionality.

The RFID Task acts as the producer task for RFID UID information. Whenever an RFID card is detected, the RFID Task extracts the RFID UID and places the information into the RFID queue using FreeRTOS queue APIs. The Processing Task acts as the consumer task and continuously waits for RFID data from the queue.

Once RFID information becomes available in the queue, the Processing Task retrieves the UID and performs attendance verification, duplicate attendance checking, master card validation, and unauthorized card detection operations.

After successful attendance verification, attendance information such as student name, RFID UID, date, and time is prepared and transmitted to the Firebase Task using another FreeRTOS queue. The Firebase Task retrieves attendance data from the queue and uploads the information to Firebase Realtime Database through WiFi communication.

Queue communication mechanisms help in decoupling tasks from each other, allowing independent task execution and improved multitasking efficiency. Tasks waiting for queue data remain in blocked state until new data becomes available, thereby improving processor utilization and reducing unnecessary CPU activity.

The queue communication architecture implemented in the proposed system provides several advantages including:

- Safe inter-task communication
- Reliable data transfer
- Efficient multitasking operation
- Improved processor utilization
- Reduced synchronization complexity
- Independent task execution
- Organized data management
- Concurrent operation handling

The proposed system mainly utilizes two FreeRTOS queues:

- RFID Queue
Used for transferring RFID UID information from RFID Task to Processing Task.
- Firebase Queue
Used for transferring attendance information from Processing Task to Firebase Task.

Queue communication also improves system scalability because additional tasks and functionalities can be integrated easily without modifying the complete system architecture.

The implemented queue-based inter-task communication mechanism demonstrates efficient utilization of FreeRTOS communication features for reliable concurrent task execution in real-time embedded attendance monitoring applications.

3.8 Mutex Synchronization Mechanism

Mutex synchronization is an important Real-Time Operating System mechanism used for protecting shared resources from simultaneous access by multiple concurrent tasks. In the proposed FreeRTOS-Based Smart Attendance Monitoring System, mutex synchronization is implemented to ensure safe and reliable operation of shared hardware peripherals such as the OLED display module.

In multitasking embedded systems, multiple FreeRTOS tasks may attempt to access the same hardware resource at the same time. Without proper synchronization mechanisms, simultaneous access may lead to data corruption, unstable system behavior, incorrect display outputs, and communication conflicts.

The proposed system implements FreeRTOS mutex mechanisms to prevent such resource conflicts and ensure proper coordination between concurrent tasks. The OLED display module is shared among multiple tasks for displaying attendance messages, attendance session status, duplicate attendance warnings, and unauthorized card alerts.

Whenever a task needs to access the OLED display, it first requests ownership of the mutex using FreeRTOS mutex APIs. If the mutex is available, the task acquires exclusive access to the display resource and performs the required display operation. During this period, other tasks attempting to access the display are temporarily blocked until the mutex is released.

After completing the display operation, the task releases the mutex and allows other waiting tasks to access the shared resource safely. This mechanism ensures that only one task can access the OLED display at any given time, preventing overlapping display operations and inconsistent output generation.

The mutex synchronization mechanism also helps in avoiding priority inversion problems in multitasking systems. FreeRTOS mutexes support priority inheritance techniques, which temporarily elevate the priority of lower-priority tasks holding the mutex when higher-priority tasks are waiting for the same resource.

The mutex synchronization architecture implemented in the proposed system provides several advantages including:

- Protection of shared resources
- Prevention of simultaneous resource access
- Improved system stability
- Reliable OLED display communication
- Reduced data corruption

- Better task coordination
- Safe multitasking operation
- Efficient synchronization management

The major mutex operations implemented in the proposed system are listed below:

- Mutex acquisition before OLED access
- Exclusive display communication
- Mutex release after operation completion
- Resource synchronization between concurrent tasks

The implementation of mutex synchronization significantly improves the reliability and stability of the proposed attendance monitoring system by ensuring safe access to shared hardware resources during concurrent task execution.

3.9 Timer Task and Scheduling Operation

The Timer Task and scheduling operation play an important role in the proposed FreeRTOS-Based Smart Attendance Monitoring System for achieving efficient multitasking, periodic monitoring, and real-time task execution management. FreeRTOS scheduling mechanisms ensure proper execution of multiple concurrent tasks based on task priority and system requirements.

The proposed system implements a preemptive priority-based scheduling mechanism provided by FreeRTOS. In this scheduling approach, higher-priority tasks are executed before lower-priority tasks, ensuring efficient real-time response and proper processor utilization.

The RFID Task is assigned the highest priority because RFID card detection and attendance acquisition require immediate processing for real-time operation. The Processing Task operates with medium priority since it performs attendance validation and authorization checking after RFID detection. The Firebase Task is assigned lower priority because cloud synchronization can occur after attendance processing is completed. The Timer Task operates with the lowest priority because it mainly performs periodic monitoring and scheduling analysis operations.

The FreeRTOS scheduler continuously monitors the status of all tasks and allocates processor time based on task priority and readiness. Whenever a higher-priority task becomes ready for execution, the currently executing lower-priority task is preempted, and processor control is transferred to the higher-priority task. This scheduling mechanism improves responsiveness and enables efficient handling of real-time operations.

The Timer Task is implemented using FreeRTOS software timer mechanisms for periodic execution of monitoring operations. The timer periodically generates RTOS scheduling traces and task execution information used for scheduling analysis and visualization through Serial Studio.

Whenever the software timer expires, the timer callback function is executed automatically by FreeRTOS. The callback function transmits task execution traces through UART serial communication for graphical scheduling analysis and RTOS visualization.

The scheduling operation also utilizes task delay mechanisms and blocked task states for improving processor utilization. Tasks waiting for queue data or timer events remain in blocked state until the required event becomes available. This prevents unnecessary processor consumption and improves multitasking efficiency.

The Timer Task and scheduling architecture implemented in the proposed system provide several advantages including:

- Efficient multitasking capability
- Priority-based task execution
- Improved processor utilization
- Faster real-time response
- Periodic system monitoring
- Reliable concurrent execution
- Reduced processor overhead
- Real-time scheduling analysis

The scheduling operation implemented using FreeRTOS demonstrates practical utilization of RTOS concepts such as preemptive scheduling, task prioritization, software timers, periodic execution, and multitasking management within the proposed attendance monitoring system.

3.10 Firebase Cloud Integration

Firebase cloud integration is an important feature implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for real-time attendance storage, remote monitoring, and cloud-based data management. The system utilizes Firebase Realtime Database for storing attendance information securely through WiFi communication using the ESP32 microcontroller.

The ESP32 microcontroller contains an integrated WiFi module that enables wireless communication with cloud platforms and IoT services. In the proposed system, WiFi connectivity is established between ESP32 and Firebase Realtime Database for transmitting attendance records to the cloud server.

After attendance verification is completed by the Processing Task, attendance details such as student name, RFID UID, date, and time are transferred to the Firebase Task through FreeRTOS queue communication. The Firebase Task independently handles cloud synchronization operations without interrupting other real-time attendance tasks.

The Firebase Task formats attendance information into structured data format and uploads the information to Firebase Realtime Database using HTTP-based cloud communication protocols. The uploaded attendance records are stored under specific database paths for easy retrieval and management.

Firebase cloud integration provides several advantages including:

- Real-time attendance storage
- Remote attendance monitoring
- Cloud-based database management
- Wireless attendance synchronization
- Secure attendance record maintenance
- Improved accessibility
- Reduced manual data handling
- Efficient IoT integration

The cloud-based attendance architecture allows attendance information to be accessed remotely through web applications, mobile applications, or cloud dashboards. This improves attendance monitoring flexibility and enables centralized attendance management for educational institutions.

The Firebase integration also improves scalability of the proposed system because large attendance records can be stored efficiently in the cloud database without requiring local storage expansion on the embedded hardware platform.

The FreeRTOS-based multitasking architecture ensures that Firebase communication operates concurrently with RFID scanning and attendance processing tasks. Queue communication mechanisms and task scheduling improve overall system responsiveness during cloud synchronization operations.

The proposed system also utilizes Firebase communication status monitoring for verifying successful attendance uploads and cloud synchronization operations. Real-time upload confirmation messages are displayed through serial monitoring and OLED notifications during attendance operations.

The Firebase cloud integration architecture demonstrates practical implementation of IoT-based attendance monitoring using embedded cloud communication, FreeRTOS multitasking, and real-time attendance synchronization mechanisms within the proposed attendance monitoring system.

3.11 Serial Studio RTOS Visualization

Serial Studio RTOS visualization is an important feature implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for real-time monitoring and graphical analysis of task execution behavior. The visualization mechanism helps in understanding FreeRTOS scheduling flow, multitasking operation, and concurrent task execution within the embedded system.

The proposed system integrates Serial Studio with ESP32 through UART serial communication. FreeRTOS tasks running on the ESP32 generate task execution traces and telemetry data during runtime. This telemetry information is transmitted to Serial Studio for graphical visualization and scheduling analysis.

Each FreeRTOS task implemented in the system is assigned a unique task identifier for monitoring purposes. The RFID Task, Processing Task, Firebase Task, and Timer Task continuously transmit task execution information through UART communication whenever they are executed by the FreeRTOS scheduler.

The transmitted telemetry data mainly includes:

- Task identification number
- Task execution timing
- Scheduling information
- RTOS execution traces
- Periodic timer activity

Serial Studio receives the telemetry information from ESP32 and displays the task execution behavior graphically using real-time plots and timing graphs. The visualization graph helps in observing task switching, scheduling order, and multitasking execution flow during attendance monitoring operations.

The RTOS visualization process begins when the ESP32 starts executing FreeRTOS tasks. As RFID cards are scanned, the RFID Task becomes active and generates task execution traces. The Processing Task executes after RFID data reception, followed by Firebase communication and periodic Timer Task execution. These scheduling events are visualized graphically in Serial Studio.

The UART-based telemetry communication implemented in the proposed system provides lightweight and efficient RTOS trace generation without significantly affecting processor performance and attendance monitoring operation.

The Serial Studio RTOS visualization architecture demonstrates practical implementation of real-time scheduling analysis and embedded multitasking monitoring within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

3.12 Working Principle

The working principle of the proposed FreeRTOS-Based Smart Attendance Monitoring System is based on real-time RFID attendance acquisition, concurrent task execution, cloud synchronization, and RTOS scheduling mechanisms implemented using FreeRTOS and ESP32 microcontroller.

The complete system operation begins when power is supplied to the ESP32 microcontroller, and all hardware peripherals are initialized. The ESP32 establishes Wi-Fi communication with Firebase Realtime Database and creates multiple FreeRTOS tasks for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring.

The RC522 RFID reader continuously scans for nearby RFID cards through the RFID Task operating under FreeRTOS. Whenever an RFID card is detected, the RFID reader extracts the RFID UID and transfers the information to the RFID Task for further processing.

The RFID Task sends the detected RFID UID to the Processing Task using FreeRTOS queue communication. The Processing Task receives the RFID information and determines whether the scanned card belongs to a master RFID card or a student RFID card.

The master RFID card is used for controlling attendance sessions. When the master card is scanned, the system either activates or terminates the attendance session depending on the current attendance state. Attendance marking is allowed only during active attendance sessions.

When a valid student RFID card is scanned during an active attendance session, the Processing Task verifies whether the RFID card is authorized and checks whether attendance has already been marked previously. If duplicate attendance is detected, the system generates duplicate attendance warnings through OLED display notifications and buzzer indications.

For valid attendance operations, student attendance information including student name, RFID UID, date, and time is recorded and transferred to the Firebase Task using queue communication mechanisms. The Firebase Task uploads attendance information to Firebase Realtime Database through WiFi communication for cloud-based attendance storage and remote monitoring.

The OLED display module continuously provides real-time notifications such as attendance confirmation, attendance session status, duplicate attendance alerts, and unauthorized card warnings. The buzzer and LED indicators provide immediate audio-visual feedback during different attendance operations.

Mutex synchronization mechanisms ensure safe access to shared hardware resources such as OLED display communication during concurrent task execution. FreeRTOS software timers periodically generate RTOS scheduling traces and monitoring information.

Serial Studio receives UART telemetry information generated by FreeRTOS tasks and displays graphical task execution and scheduling visualization in real time. This enables analysis of multitasking behavior, scheduling flow, and processor activity within the embedded system.

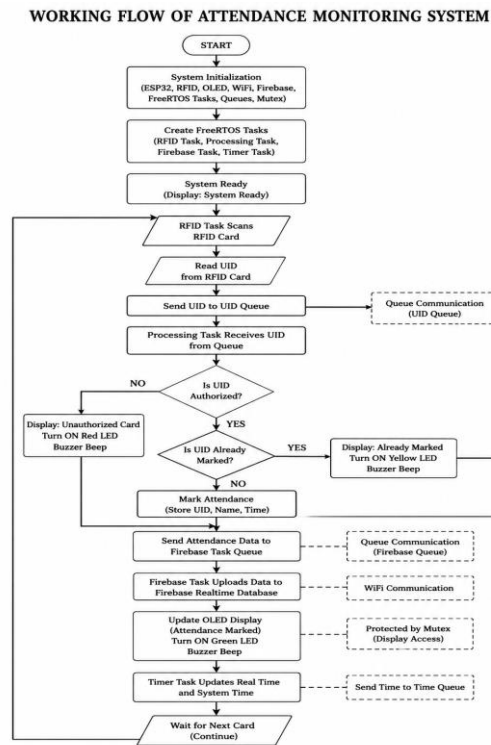


Figure 3.3: Working Flow of Attendance Monitoring System

The overall working principle demonstrates efficient integration of RFID technology, FreeRTOS multitasking, queue communication, mutex synchronization, cloud synchronization, and RTOS visualization mechanisms within the proposed smart attendance monitoring system.

3.13 Summary

This chapter presented the proposed architecture and methodology of the FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, RC522 RFID reader, Firebase cloud integration, and FreeRTOS multitasking mechanisms.

The overall system architecture, hardware-software integration, and operational workflow of the proposed system were explained in detail. The proposed system utilizes multiple FreeRTOS tasks for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring to achieve efficient multitasking and real-time attendance management.

The chapter also discussed the implementation of queue-based inter-task communication and mutex synchronization mechanisms for safe data transfer and protected resource sharing between concurrent tasks. The utilization of FreeRTOS scheduling mechanisms and software timers for periodic monitoring and efficient processor utilization was also presented.

Firebase cloud integration for real-time attendance storage and remote monitoring was explained along with WiFi communication and cloud synchronization architecture. The integration of Serial Studio for RTOS scheduling visualization and real-time task monitoring was also discussed in detail.

The proposed working principle demonstrated how RFID attendance acquisition, task scheduling, queue communication, synchronization mechanisms, cloud integration, and real-time visualization operate together within the embedded attendance monitoring system.

The overall proposed architecture provides efficient multitasking capability, reliable attendance monitoring, cloud-based attendance management, real-time scheduling analysis, and practical implementation of RTOS concepts suitable for embedded systems and IoT-based applications.

The next chapter presents the hardware implementation details, experimental setup, hardware components, interfacing mechanisms, and physical system integration of the proposed attendance monitoring system.

CHAPTER 4 HARDWARE IMPLEMENTATION

4.1 Introduction

This chapter presents the hardware implementation details of the proposed FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, and WiFi-based cloud communication architecture.

The hardware implementation focuses on integrating various embedded hardware peripherals required for real-time attendance monitoring, multitasking operation, cloud synchronization, and RTOS scheduling visualization. The ESP32 microcontroller acts as the central processing unit responsible for executing FreeRTOS tasks, handling RFID communication, managing cloud synchronization, controlling display modules, and generating RTOS telemetry traces.

The RC522 RFID reader is used for contactless RFID card detection and attendance acquisition. The OLED display module provides real-time attendance notifications and system status information. Buzzer and LED indicators are used for audio-visual feedback during attendance operations. WiFi communication capability available in the ESP32 enables Firebase cloud synchronization and remote attendance monitoring.

The hardware architecture also supports Serial Studio integration through UART communication for real-time RTOS task visualization and scheduling analysis. Proper interfacing techniques, communication protocols, power supply arrangements, and embedded hardware integration methods are implemented to ensure reliable system operation.

This chapter explains the specifications, interfacing methods, operational functionality, and hardware integration details of all major hardware components used in the proposed attendance monitoring system.

4.2 ESP32 Development Board

The ESP32 development board is the main processing unit used in the proposed FreeRTOS-Based Smart Attendance Monitoring System. It is a low-cost, high-performance microcontroller developed by Espressif Systems with integrated Wi-Fi and Bluetooth communication capability suitable for IoT and real-time embedded applications.

The ESP32 microcontroller is selected for the proposed system because of its powerful dual-core processor architecture, multitasking capability, large GPIO support, integrated wireless communication, and compatibility with FreeRTOS. The ESP32 efficiently handles RFID communication, cloud synchronization, OLED display control, serial telemetry transmission, and concurrent task execution within the attendance monitoring system.

The ESP32 board operates at a clock frequency of up to 240 MHz and contains sufficient RAM and flash memory for implementing multiple FreeRTOS tasks and cloud communication operations. The microcontroller supports various communication interfaces such as SPI, I2C, UART, PWM, ADC, and WiFi communication, making it suitable for interfacing multiple hardware peripherals simultaneously.

In the proposed system, the ESP32 performs the following major operations:

- RFID card detection and processing
- FreeRTOS task scheduling
- Queue communication management
- Mutex synchronization handling
- Firebase cloud communication
- OLED display interfacing
- UART telemetry transmission
- Serial Studio integration
- Attendance data processing
- Buzzer and LED control

The ESP32 communicates with the RC522 RFID reader through SPI communication protocol. The OLED display module is interfaced using I2C communication. UART communication is utilized for transmitting RTOS telemetry data to Serial Studio for scheduling visualization and task monitoring.

The integrated WiFi module available in ESP32 enables wireless communication with Firebase Realtime Database for cloud-based attendance synchronization and remote attendance monitoring.

The ESP32 board also supports FreeRTOS natively, allowing implementation of multitasking architecture, task scheduling, inter-task communication, synchronization mechanisms, and software timers required for real-time embedded operation.

The major features of ESP32 used in the proposed system are listed below:

- Dual-core processor architecture
- Integrated WiFi communication
- FreeRTOS support
- Multiple GPIO pins
- SPI, I2C, and UART interfaces
- High processing speed
- Low power consumption
- Embedded multitasking capability
- Real-time processing support

The ESP32 development board provides reliable performance, efficient multitasking operation, and seamless hardware-software integration within the proposed smart attendance monitoring system.

4.3 RC522 RFID Reader

The RC522 RFID reader is an important hardware component used in the proposed FreeRTOS-Based Smart Attendance Monitoring System for contactless RFID card detection and attendance acquisition. The RFID reader is responsible for reading RFID card information and transmitting the detected UID to the ESP32 microcontroller for attendance processing.

The RC522 module operates based on Radio Frequency Identification technology using electromagnetic fields for wireless communication between RFID tags and the RFID reader. Each RFID card contains a unique identification number known as UID, which is used for student authentication and attendance verification.

The RC522 RFID reader operates at a frequency of 13.56 MHz and supports SPI communication protocol for interfacing with microcontrollers such as ESP32. The module provides fast RFID card detection, low power consumption, reliable communication, and efficient contactless operation suitable for attendance monitoring applications.

In the proposed system, the RC522 RFID reader continuously scans for nearby RFID cards using the RFID Task implemented under FreeRTOS. Whenever an RFID card is brought near the reader, the module detects the RFID UID and transfers the information to the ESP32 microcontroller through SPI communication.

The ESP32 processes the received RFID UID and determines whether the scanned card belongs to a master RFID card or a student RFID card. The master card is used for attendance session control, while student cards are used for attendance marking.

The RC522 RFID reader is interfaced with ESP32 using SPI communication pins. The hardware connections implemented in the proposed system are listed below:

RC522 Pin	ESP32 Pin
SDA	GPIO 5
SCK	GPIO 18
MOSI	GPIO 23
MISO	GPIO 19
RST	GPIO 4
VCC	3.3V
GND	GND

The RFID reader provides several advantages in the proposed attendance monitoring system including:

- Contactless attendance acquisition

- Fast RFID card detection
- Low power consumption
- Reliable communication
- Easy hardware interfacing
- Reduced manual effort
- Efficient student identification
- Real-time attendance monitoring

The RFID reader operates continuously during attendance sessions and supports real-time RFID scanning under FreeRTOS multitasking architecture. Queue communication mechanisms are used for transferring RFID UID information from the RFID Task to the Processing Task safely and efficiently.

The RC522 RFID reader significantly improves automation capability and attendance acquisition efficiency within the proposed smart attendance monitoring system.

4.4 OLED Display Module

The OLED display module is used in the proposed FreeRTOS-Based Smart Attendance Monitoring System for displaying real-time attendance notifications, system status information, and user interaction messages. The display provides visual feedback during attendance operations and improves the usability of the embedded attendance monitoring system.

The proposed system utilizes a 128×64 SSD1306 OLED display module interfaced with the ESP32 microcontroller using the I2C communication protocol. OLED technology provides high contrast ratio, low power consumption, faster response time, and better visibility compared to conventional LCD displays.

The OLED display is responsible for displaying various system messages such as:

- Attendance confirmation
- Attendance session status
- Student identification
- Duplicate attendance warning
- Unauthorized card detection
- System initialization messages
- Cloud synchronization status

When an RFID card is scanned, the Processing Task verifies the attendance information and updates the OLED display with the corresponding attendance status. Valid attendance entries

display the student's name and attendance confirmation message. Duplicate attendance attempts and invalid RFID cards to generate warning messages on the display.

The OLED display module communicates with ESP32 using I2C communication pins. The hardware connections implemented in the proposed system are listed below:

OLED Pin	ESP32 Pin
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Since multiple FreeRTOS tasks may attempt to access the OLED display simultaneously, mutex synchronization mechanisms are implemented for safe display communication. Before accessing the OLED display, a task acquires the mutex to ensure exclusive access to the display resource. After completing the display operation, the mutex is released for other tasks.

The OLED display module provides several advantages in the proposed system including:

- Real-time user notifications
- High display clarity
- Fast response time
- Low power consumption
- Compact size
- Easy I2C interfacing
- Reliable visual feedback
- Improved user interaction

The OLED display continuously updates system information during attendance operation and provides effective visual monitoring capability for users interacting with the attendance system.

The implementation of the OLED display module improves system usability, user communication, and real-time status monitoring within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

4.5 Buzzer and LED Indicators

The buzzer and LED indicators are used in the proposed FreeRTOS-Based Smart Attendance Monitoring System for providing audio and visual feedback during attendance operations. These components improve user interaction, system monitoring, and real-time notification capability within the embedded attendance monitoring architecture.

The buzzer module generates audio alerts for different attendance events such as successful attendance marking, duplicate attendance detection, invalid RFID card access, and attendance session control operations. The audio indications help users identify system status and attendance responses without continuously observing the OLED display.

LED indicators are implemented for visual status monitoring of system operations. The proposed system utilizes separate LEDs for indicating successful attendance operation and warning conditions. The green LED is used for successful attendance confirmation and attendance session activation, while the red LED is used for invalid card detection, duplicate attendance warning, and attendance session termination indication.

Whenever a valid RFID card is scanned during an active attendance session, the Processing Task activates the green LED and buzzer for a short duration to indicate successful attendance marking. If duplicate attendance or unauthorized RFID access is detected, the red LED and buzzer generate warning indications.

The buzzer and LED indicators are directly controlled by the ESP32 microcontroller through GPIO pins. The hardware connections implemented in the proposed system are listed below:

Component	ESP32 Pin
Buzzer	GPIO 27
Green LED	GPIO 14
Red LED	GPIO 12

The buzzer is interfaced using an NPN transistor driver circuit to provide sufficient current amplification and safe operation. The transistor-based driver protects the ESP32 GPIO pins from excessive current during buzzer activation.

The buzzer and LED indication system provides several advantages including:

- Real-time audio feedback
- Immediate visual indication
- Improved user interaction
- Easy attendance status monitoring
- Fast response indication
- Efficient warning notification
- Reduced dependency on display observation
- Better system usability

The buzzer and LED indicators operate concurrently with other FreeRTOS tasks under the multitasking architecture. The Processing Task manages activation and deactivation of audio-visual indications based on attendance verification results and system conditions.

The implementation of buzzer and LED indicators improves real-time interaction capability, user experience, and operational monitoring within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

4.6 WiFi Communication Module

The WiFi communication module plays an important role in the proposed FreeRTOS-Based Smart Attendance Monitoring System for enabling wireless cloud communication and real-time attendance synchronization. The ESP32 microcontroller used in the system contains an integrated WiFi module that provides reliable wireless connectivity for IoT-based applications and cloud communication.

The integrated WiFi capability of ESP32 eliminates the need for external wireless communication hardware and simplifies the overall system architecture. The WiFi module is used for establishing communication between the ESP32 microcontroller and Firebase Realtime Database for cloud-based attendance monitoring and remote accessibility.

In the proposed system, the ESP32 connects to a wireless network using predefined WiFi credentials such as network SSID and password. After successful network connection, the Firebase Task establishes communication with Firebase Realtime Database using internet connectivity provided through the WiFi module.

Whenever attendance is successfully verified, attendance information including student name, RFID UID, date, and time is transmitted to Firebase cloud database through WiFi communication. The attendance data is uploaded in real time, enabling remote attendance monitoring and cloud-based attendance management.

The WiFi communication module supports concurrent operation under the FreeRTOS multitasking architecture. While one task handles RFID scanning and attendance processing, the Firebase Task independently manages WiFi communication and cloud synchronization without interrupting other system operations.

The integrated WiFi communication architecture provides several advantages including:

- Wireless attendance synchronization
- Real-time cloud communication
- Remote attendance monitoring
- Reduced hardware complexity
- Efficient IoT integration
- Faster data transmission
- Cloud-based attendance storage

- Improved system scalability

The WiFi module also enables easy integration with mobile applications, cloud dashboards, and web-based attendance management systems for future system expansion.

The proposed system continuously monitors WiFi connectivity status to ensure reliable cloud communication during attendance operations. If network connectivity is interrupted, the system attempts reconnection to maintain stable communication with the Firebase cloud platform.

The implementation of WiFi communication significantly improves automation capability, remote accessibility, and cloud integration efficiency within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

4.7 Power Supply Arrangement

The power supply arrangement is an important part of the proposed FreeRTOS-Based Smart Attendance Monitoring System because all hardware modules require stable and reliable power for proper operation. The proposed system is designed to operate using low-voltage DC power suitable for embedded and IoT-based applications.

The ESP32 development board acts as the main power distribution unit for the system. The board can be powered through USB connection or external 5V DC supply. Internally, the ESP32 board regulates the supplied voltage and provides required operating voltages for connected peripherals.

The RC522 RFID reader and OLED display module operates at 3.3V supply voltage and are powered directly from the ESP32 development board. Proper voltage matching is maintained to avoid hardware damage and communication instability. The ESP32 GPIO pins also operate at 3.3V logic levels, ensuring safe interfacing with the RFID reader and OLED display.

The buzzer and LED indicators are powered using GPIO-controlled output pins of the ESP32. The buzzer is interfaced using an NPN transistor driver circuit for current amplification and reliable switching operation. Current-limiting resistors are connected with LEDs to prevent excessive current flow and protect the components from damage.

The overall power distribution architecture ensures stable voltage supply to all hardware peripherals during simultaneous operation of RFID scanning, OLED display communication, WiFi transmission, and FreeRTOS multitasking execution.

The power supply arrangement implemented in the proposed system provides several advantages including:

- Stable hardware operation
- Reliable peripheral communication
- Low power consumption
- Safe voltage regulation
- Compact embedded architecture
- Efficient IoT device operation
- Reduced hardware complexity
- Portable system implementation

The ESP32 development board efficiently manages power distribution for all connected modules while supporting WiFi communication, FreeRTOS task execution, cloud synchronization, and UART telemetry transmission simultaneously.

Proper grounding connections are maintained between all hardware modules to ensure noise-free communication and stable embedded system performance. The overall power supply architecture improves reliability, stability, and continuous operation of the proposed smart attendance monitoring system.

4.8 Hardware Connections and Experimental Setup

The hardware connections and experimental setup of the proposed FreeRTOS-Based Smart Attendance Monitoring System were designed to ensure reliable communication between all embedded peripherals and stable real-time system operation. The complete setup integrates ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, WiFi communication, and Serial Studio visualization interface.

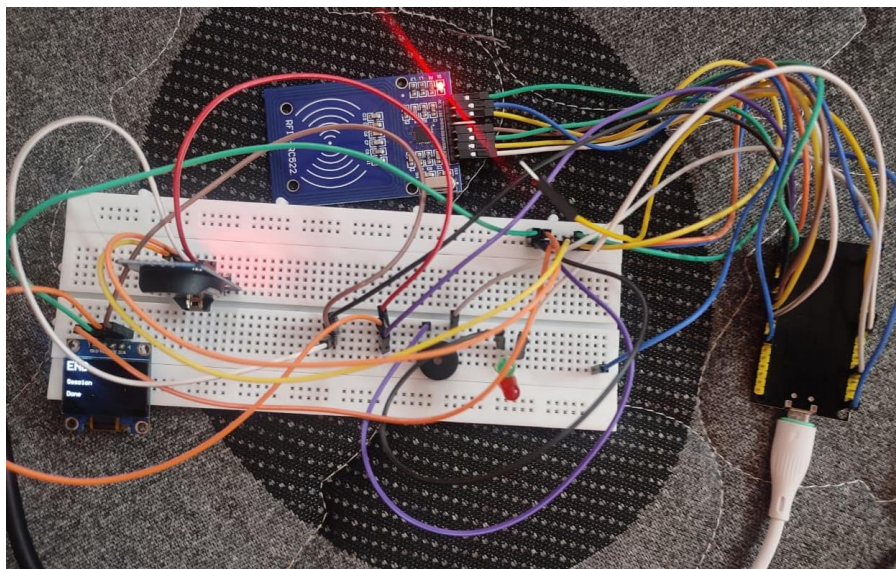


Figure 4.1: Hardware Setup of the Attendance Monitoring System

The ESP32 development board acts as the central controller and interfaces with all hardware modules through SPI, I2C, GPIO, and UART communication protocols. Proper hardware interfacing and wiring arrangements were implemented to achieve stable attendance monitoring and multitasking execution under FreeRTOS.

The RC522 RFID reader is connected to the ESP32 using SPI communication protocol. The RFID reader continuously scans RFID cards and transmits RFID UID information to the ESP32 for attendance processing. The OLED display module is connected using the I2C communication interface and is used for displaying attendance notifications and system messages.

The buzzer and LED indicators are interfaced through GPIO pins of ESP32 for generating audio-visual attendance feedback. The buzzer is connected through an NPN transistor driver circuit for current amplification and reliable operation. The ESP32 integrated WiFi module enables wireless communication with Firebase Realtime Database for cloud-based attendance synchronization.

UART serial communication is established between ESP32 and the computer system for transmitting RTOS telemetry traces to Serial Studio. Serial Studio visualizes task scheduling behavior and multitasking execution flow in graphical form during real-time attendance operation.

The complete hardware connections used in the proposed system are summarized below.

Hardware Component	ESP32 Pin Connection
RC522 SDA	GPIO 5
RC522 SCK	GPIO 18
RC522 MOSI	GPIO 23
RC522 MISO	GPIO 19
RC522 RST	GPIO 4
OLED SDA	GPIO 21
OLED SCL	GPIO 22
Buzzer	GPIO 27
Green LED	GPIO 14
Red LED	GPIO 12

The experimental setup was assembled on a breadboard platform using jumper wire connections for easy hardware interfacing and testing. USB communication was used for powering the ESP32 board and for UART telemetry transmission during RTOS scheduling visualization.

During system operation, RFID cards were scanned using the RC522 reader to verify attendance functionality, queue communication, multitasking operation, Firebase cloud synchronization, and

RTOS scheduling visualization. Serial Studio displayed real-time task execution graphs generated through FreeRTOS telemetry traces transmitted from the ESP32.

The experimental setup successfully demonstrated:

- RFID-based attendance acquisition
- Real-time FreeRTOS multitasking
- Queue-based inter-task communication
- Mutex synchronization
- Firebase cloud synchronization
- OLED display notifications
- Audio-visual attendance feedback
- RTOS scheduling visualization
- Real-time telemetry monitoring

The implemented hardware setup provided stable operation, reliable attendance monitoring, efficient multitasking execution, and successful integration of RTOS concepts within the proposed embedded attendance monitoring system.

4.9 Summary

This chapter presented the hardware implementation details and experimental setup of the proposed FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, WiFi communication, and Firebase cloud integration.

The chapter explained the specifications, functionality, interfacing methods, and operational role of each hardware component used in the system. The ESP32 development board was utilized as the central processing unit for executing FreeRTOS tasks, handling communication protocols, and managing real-time attendance operations.

The RC522 RFID reader was used for contactless attendance acquisition, while the OLED display module provided real-time attendance notifications and system status messages. Buzzer and LED indicators were implemented for audio-visual attendance feedback and warning indications.

The integrated WiFi communication capability of ESP32 enabled cloud synchronization with Firebase Realtime Database for remote attendance monitoring and cloud-based attendance storage. UART communication was used for transmitting RTOS telemetry traces to Serial Studio for real-time scheduling visualization and multitasking analysis.

The hardware connections, power supply arrangement, and experimental setup were also discussed in detail. Proper interfacing techniques and communication protocols were implemented to ensure stable operation and reliable multitasking execution under FreeRTOS architecture.

The experimental setup successfully demonstrated RFID attendance monitoring, queue communication, mutex synchronization, Firebase cloud synchronization, RTOS scheduling visualization, and concurrent task execution using FreeRTOS.

The next chapter presents the software implementation details, FreeRTOS task programming, scheduling logic, queue communication algorithms, synchronization mechanisms, Firebase software integration, and RTOS telemetry generation implemented in the proposed attendance monitoring system.

CHAPTER 5 SOFTWARE IMPLEMENTATION

5.1 Introduction

This chapter presents the software implementation details of the proposed FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, FreeRTOS multitasking mechanisms, RFID communication, Firebase cloud synchronization, and Serial Studio telemetry visualization.

The software implementation focuses on developing efficient real-time embedded applications capable of performing concurrent task execution, attendance processing, cloud communication, synchronization management, and RTOS scheduling analysis. The entire software architecture is implemented using Arduino IDE with FreeRTOS support for ESP32.

The proposed system utilizes multiple FreeRTOS tasks for handling RFID scanning, attendance validation, Firebase communication, OLED display handling, and timer-based monitoring. Queue communication mechanisms and mutex synchronization techniques are implemented to ensure safe data transfer and protected resource sharing between concurrent tasks.

The software implementation also integrates Firebase Realtime Database for cloud-based attendance storage and remote monitoring through WiFi communication. UART telemetry communication is utilized for transmitting RTOS scheduling traces and task execution information to Serial Studio for graphical scheduling visualization.

This chapter explains the development environment, FreeRTOS task implementation, queue communication logic, synchronization mechanisms, scheduling operations, Firebase software

integration, OLED interfacing, and RTOS telemetry generation implemented within the proposed attendance monitoring system.

5.2 Arduino IDE Development Environment

The Arduino IDE development environment is used for software development and programming of the proposed FreeRTOS-Based Smart Attendance Monitoring System. Arduino IDE provides a simple and efficient platform for developing embedded applications using ESP32 microcontroller and FreeRTOS libraries.

Arduino IDE is an open-source integrated development environment widely used for programming microcontrollers and IoT-based embedded systems. It supports multiple hardware platforms and provides built-in libraries for communication protocols, sensor interfacing, WiFi communication, and real-time embedded programming.

In the proposed system, Arduino IDE is used for implementing FreeRTOS multitasking architecture, RFID communication, Firebase cloud synchronization, OLED display interfacing, queue communication, mutex synchronization, timer management, and UART telemetry generation.

The ESP32 board package is installed in the Arduino IDE to enable ESP32 programming support and FreeRTOS integration. Various embedded libraries required for hardware interfacing and cloud communication are included in the development environment.

The major libraries used in the proposed system are listed below:

Library	Purpose
SPI Library	RC522 RFID communication
MFRC522 Library	RFID card reading
Wire Library	I2C communication
Adafruit SSD1306	OLED display control
Adafruit GFX	Graphics support
WiFi Library	WiFi communication
FirebaseESP32 Library	Firebase cloud integration
FreeRTOS Libraries	Multitasking and scheduling

The Arduino IDE development environment provides several important features for embedded software development including:

- Source code editing
- Embedded library management
- Serial monitoring

- Real-time debugging support
- ESP32 program uploading
- FreeRTOS support
- UART communication monitoring
- IoT application development

The software development process begins with initialization of hardware peripherals such as RFID readers, OLED displays, buzzers, LEDs, WiFi communication, and Firebase connectivity. After initialization, multiple FreeRTOS tasks are created using FreeRTOS APIs for handling concurrent attendance operations.

The Arduino IDE serial monitor is also used for debugging and monitoring telemetry data generated during RTOS scheduling visualization. UART telemetry traces transmitted from ESP32 are visualized using Serial Studio for analyzing multitasking behavior and task scheduling flow.

The Arduino IDE development environment provides an efficient and flexible platform for implementing real-time embedded attendance monitoring applications using ESP32 and FreeRTOS multitasking architecture.

5.3 FreeRTOS Task Implementation

FreeRTOS task implementation is the core software architecture of the proposed FreeRTOS-Based Smart Attendance Monitoring System. Multiple FreeRTOS tasks are created for handling RFID scanning, attendance processing, Firebase communication, and timer-based monitoring concurrently within the embedded system.

FreeRTOS tasks are independent execution units that operate concurrently under the control of the FreeRTOS scheduler. Each task performs a specific operation and executes continuously based on task priority and scheduling conditions. The multitasking architecture implemented in the proposed system improves processor utilization, system responsiveness, and real-time performance.

The proposed system implements the following major FreeRTOS tasks:

- RFID Task
- Processing Task
- Firebase Task
- Timer Task

Each task is created using the `xTaskCreate()` FreeRTOS API and assigned a specific priority based on its operational importance. The RFID Task is assigned the highest priority because RFID card detection requires immediate response during attendance operations. The Processing Task operates with medium priority, while the Firebase Task and Timer Task operate with lower priorities.

The RFID Task continuously scans RFID cards using the RC522 RFID reader and extracts RFID UID information whenever a valid card is detected. The detected RFID UID is transferred to the Processing Task using queue communication mechanisms.

The Processing Task performs attendance verification, master card authentication, duplicate attendance detection, and unauthorized RFID card checking. Attendance information generated by this task is transmitted to the Firebase Task through another queue.

The Firebase Task handles WiFi communication and uploads attendance information to Firebase Realtime Database for cloud synchronization and remote monitoring.

The Timer Task periodically generates RTOS telemetry traces and scheduling information used for real-time task visualization through Serial Studio.

The basic structure of FreeRTOS task creation implemented in the proposed system is shown below:

```
xTaskCreate(  
    RFID_Task,  
    "RFID Task",  
    4096,  
    NULL,  
    3,  
    NULL  
);
```

In the above implementation:

- `RFID_Task` represents the task function
- `"RFID Task"` represents the task name
- `4096` represents task stack size
- `3` represents task priority

The FreeRTOS scheduler continuously manages execution of all created tasks based on task priority and readiness. Higher-priority tasks preempt lower-priority tasks whenever required, ensuring efficient real-time operation.

The FreeRTOS task implementation provides several advantages including:

- Concurrent task execution
- Efficient multitasking capability
- Improved processor utilization
- Faster real-time response
- Independent task operation
- Better scheduling efficiency
- Modular software architecture
- Reliable embedded system operation

The implemented multitasking architecture demonstrates practical utilization of FreeRTOS task management mechanisms for real-time embedded attendance monitoring applications.

5.4 RFID Task Implementation

The RFID Task is one of the most important FreeRTOS tasks implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System. This task is responsible for continuously scanning RFID cards, detecting RFID UID information, and transferring the acquired attendance data to other tasks for further processing.

The RFID Task operates continuously under the control of the FreeRTOS scheduler and is assigned the highest task priority because RFID detection requires immediate real-time response during attendance monitoring operations.

The RC522 RFID reader is interfaced with the ESP32 microcontroller using SPI communication protocol. The RFID Task continuously checks for the presence of nearby RFID cards using RFID library functions. Whenever an RFID card is detected, the RFID Task reads the RFID UID and converts it into a suitable format for attendance processing.

The RFID UID information extracted by the RFID Task is transferred to the Processing Task using FreeRTOS queue communication mechanisms. Queue communication ensures safe and reliable transfer of RFID information between concurrent tasks without synchronization conflicts.

The RFID Task continuously executes inside an infinite loop and periodically checks RFID card availability. If no RFID card is detected, the task enters delay state using FreeRTOS delay mechanisms to reduce unnecessary processor utilization.

The basic RFID detection logic implemented in the proposed system is shown below:

```
if (!mfr522.PICC_IsNewCardPresent()) {  
    vTaskDelay(100 / portTICK_PERIOD_MS);  
    continue;  
}
```

When a valid RFID card is detected, the RFID UID is extracted using the RFID library and converted into string format for attendance processing.

The RFID Task also generates RTOS telemetry traces for Serial Studio visualization. Whenever the task executes, task execution information and timing traces are transmitted through UART serial communication for real-time scheduling analysis.

The telemetry generation logic implemented inside the RFID Task is shown below:

```
Serial.print("1,");  
Serial.println(millis());
```

In the telemetry output:

- 1 represents RFID Task identifier
- millis() represents current system timing information

The RFID Task implementation provides several advantages including:

- Real-time RFID detection
- Fast attendance acquisition
- Efficient multitasking operation
- Reliable RFID communication
- Reduced processor overhead
- Concurrent task execution
- RTOS scheduling visualization
- Improved attendance monitoring efficiency

The RFID Task continuously operates during attendance sessions and plays an important role in real-time attendance acquisition and embedded multitasking execution within the proposed attendance monitoring system.

5.5 Attendance Processing Task

The Attendance Processing Task is responsible for validating RFID information, controlling attendance sessions, verifying student attendance, detecting duplicate attendance entries, and

managing unauthorized RFID card access within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

The Processing Task receives RFID UID information from the RFID Task through FreeRTOS queue communication mechanisms. After receiving the RFID UID, the task performs various attendance verification operations based on predefined attendance logic and system conditions.

The Processing Task operates with medium priority under the FreeRTOS scheduler because it performs important attendance verification and decision-making operations immediately after RFID detection.

The attendance processing operation begins when RFID UID information becomes available in the RFID queue. The Processing Task retrieves the RFID UID using FreeRTOS queue APIs and checks whether the scanned card belongs to a master RFID card or a student RFID card.

The master RFID card is used for controlling attendance sessions. When the master card is scanned, the Processing Task either activates or terminates the attendance session depending on the current attendance state.

The attendance session control logic implemented in the proposed system is shown below:

```
if (uid == masterUID) {  
    attendanceSession = !attendanceSession;  
}
```

During active attendance sessions, student RFID cards are accepted for attendance marking. The Processing Task verifies whether the student's RFID card is authorized and checks whether attendance has already been marked previously.

If duplicate attendance is detected, the system displays duplicate attendance warning messages through the OLED display and generates warning indications using the buzzer and red LED.

For valid attendance operations, attendance information including student name, RFID UID, and timing information is recorded and transferred to the Firebase Task through FreeRTOS to queue communication for cloud synchronization.

The Processing Task also handles unauthorized RFID card detection. If an invalid RFID card is scanned, the system generates warning notifications through OLED display messages, buzzer alerts, and LED indications.

Mutex synchronization mechanisms are implemented for safe OLED display communication during concurrent task execution. Before updating the display, the Processing Task acquires the display mutex to prevent simultaneous access conflicts with other tasks.

The Processing Task also generates RTOS telemetry traces for Serial Studio visualization and scheduling analysis. Task execution timing information is transmitted through UART communication whenever the task is executed.

The telemetry generation logic implemented in the Processing Task is shown below:

```
Serial.print("2,");  
Serial.println(millis());
```

In the telemetry output:

- 2 represents Processing Task identifier
- millis() represents current execution timing

The Attendance Processing Task provides several advantages including:

- Real-time attendance verification
- Duplicate attendance prevention
- Unauthorized card detection
- Attendance session control
- Reliable multitasking operation
- Efficient queue communication
- Real-time user notification
- RTOS scheduling visualization

The Processing Task plays an important role in ensuring reliable attendance management, authorization handling, and concurrent task coordination within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

5.6 Firebase Communication Task

The Firebase Communication Task is responsible for cloud synchronization and remote attendance data storage in the proposed FreeRTOS-Based Smart Attendance Monitoring System. This task independently manages WiFi communication and Firebase database operations using FreeRTOS multitasking architecture.

The Firebase Task operates concurrently with other FreeRTOS tasks such as RFID scanning and attendance processing without interrupting real-time attendance acquisition. The task is assigned lower priority because cloud synchronization operations are less time-critical compared to RFID detection and attendance verification.

Attendance information generated by the Processing Task is transferred to the Firebase Task using FreeRTOS queue communication mechanisms. The Firebase Task continuously waits for attendance data in the Firebase queue and uploads the received information to Firebase Realtime Database through WiFi communication.

The attendance information uploaded to the cloud database mainly includes:

- Student name
- RFID UID
- Attendance timing
- Attendance status

The ESP32 integrated WiFi module establishes communication with Firebase cloud platform using predefined WiFi credentials and Firebase authentication parameters. Once the ESP32 connects to the wireless network, the Firebase Task initializes cloud communication and maintains synchronization with Firebase Realtime Database.

The Firebase data upload operation implemented in the proposed system is shown below:

```
Firestore.setString(fbdo, path + "/UID", fbData.uid);
```

```
Firestore.setInt(fbdo, path + "/TIME", millis());
```

In the above implementation:

- fbdo represents Firebase data object
- path represents database storage location
- fbData.uid represents RFID UID information
- millis() stores attendance timing information

The Firebase Task continuously monitors cloud communication status and performs attendance upload operations whenever valid attendance information becomes available in the Firebase queue.

The task also generates RTOS telemetry traces for Serial Studio visualization and scheduling analysis. Task execution information is transmitted through UART communication during Firebase synchronization operations.

The telemetry generation logic implemented inside the Firebase Task is shown below:

```
Serial.print("3,");  
Serial.println(millis());
```

In the telemetry output:

- 3 represents Firebase Task identifier
- millis() represents current execution timing

The Firebase Communication Task provides several advantages including:

- Real-time cloud synchronization
- Remote attendance monitoring
- Concurrent WiFi communication
- Efficient multitasking operation
- Reliable attendance storage
- Cloud-based database management
- Improved system scalability
- RTOS scheduling visualization

The Firebase Task improves automation capability and enables cloud-based attendance management while maintaining efficient multitasking performance within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

5.7 Queue Communication Logic

Queue communication logic is an important software mechanism implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for safe and reliable inter-task data transfer between concurrent FreeRTOS tasks. The queue communication architecture enables efficient multitasking and synchronized operation of RFID scanning, attendance processing, and Firebase communication tasks.

In FreeRTOS-based systems, queues are commonly used for exchanging information between producer tasks and consumer tasks without direct dependency between tasks. The proposed system utilizes queue communication for transferring RFID UID information and attendance data safely between different tasks operating concurrently.

The proposed architecture mainly implements two FreeRTOS queues:

- RFID Queue
- Firebase Queue

The RFID Queue is used for transferring RFID UID information from the RFID Task to the Processing Task. Whenever an RFID card is detected, the RFID Task extracts the RFID UID and places the data into the RFID Queue using FreeRTOS queue APIs.

The Processing Task continuously waits for RFID information from the queue. Once data becomes available, the Processing Task retrieves the RFID UID and performs attendance verification, authorization checking, and duplicate attendance detection.

The queue transmission operation implemented in the RFID Task is shown below:

```
xQueueSend(rfidQueue, &data, portMAX_DELAY);
```

In the above implementation:

- rfidQueue represents RFID communication queue
- data contains RFID UID information
- portMAX_DELAY blocks the task until queue space becomes available

The Processing Task receives RFID information from the queue using the following queue reception logic:

```
xQueueReceive(rfidQueue, &receivedData, portMAX_DELAY);
```

After successful attendance verification, attendance information such as student name, RFID UID, and attendance timing is transferred to the Firebase Task through the Firebase Queue.

The Firebase Queue communication logic implemented in the proposed system is shown below:

```
xQueueSend(firebaseQueue, &fbData, portMAX_DELAY);
```

The Firebase Task retrieves attendance data from the queue and uploads the information to Firebase Realtime Database for cloud synchronization.

Queue communication mechanisms improve system modularity because each task operates independently without direct interference with other tasks. Tasks waiting for queue data remain in blocked state until new data becomes available, improving processor utilization and reducing unnecessary CPU activity.

The queue communication logic provides several advantages including:

- Safe inter-task communication
- Reliable data transfer
- Improved multitasking capability
- Efficient processor utilization
- Reduced synchronization complexity

- Organized task coordination
- Concurrent task execution
- Better software modularity

The queue communication architecture implemented in the proposed system demonstrates practical utilization of FreeRTOS inter-task communication mechanisms for real-time embedded attendance monitoring applications.

5.8 Mutex Synchronization Logic

Mutex synchronization logic is implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System to ensure safe access to shared hardware resources during concurrent task execution. The synchronization mechanism prevents simultaneous access conflicts and improves overall system stability and reliability under FreeRTOS multitasking architecture.

In real-time embedded systems, multiple tasks may attempt to access the same hardware resource simultaneously. Without proper synchronization mechanisms, this may lead to data corruption, unstable operation, overlapping communication, and inconsistent output generation.

The proposed system mainly utilizes mutex synchronization for protecting OLED display communication because multiple tasks such as the Processing Task and system monitoring operations may attempt to update the OLED display concurrently.

FreeRTOS mutex mechanisms provide exclusive ownership of shared resources. Before accessing the OLED display, a task must acquire the mutex using FreeRTOS synchronization APIs. Once the mutex is acquired, other tasks attempting to access the same resource are temporarily blocked until the mutex is released.

The mutex acquisition logic implemented in the proposed system is shown below:

```
if (xSemaphoreTake(displayMutex, portMAX_DELAY)) {

    display.clearDisplay();

    display.setCursor(0, 10);
    display.println("Attendance Marked");

    display.display();
```

```
xSemaphoreGive(displayMutex);  
}
```

In the above implementation:

- displayMutex represents OLED display mutex
- xSemaphoreTake() acquires mutex ownership
- xSemaphoreGive() releases the mutex
- portMAX_DELAY blocks the task until mutex becomes available

When a task acquires the mutex, it gains exclusive access to the OLED display and performs the required display operation safely. After completing the operation, the task releases the mutex and allows other waiting tasks to access the display resource.

Mutex synchronization also helps in avoiding priority inversion problems in multitasking systems. FreeRTOS mutexes support priority inheritance mechanisms, which temporarily increase the priority of lower-priority tasks holding the mutex when higher-priority tasks are waiting for the same resource.

The mutex synchronization logic provides several advantages including:

- Protection of shared resources
- Prevention of simultaneous access conflicts
- Improved system stability
- Reliable OLED communication
- Better multitasking coordination
- Reduced data corruption
- Safe concurrent execution
- Efficient resource management

The proposed system utilizes mutex synchronization to ensure stable display communication during concurrent attendance processing and multitasking operation under FreeRTOS architecture.

The implemented mutex synchronization logic demonstrates practical utilization of FreeRTOS synchronization mechanisms for reliable embedded system operation and safe resource sharing within the proposed attendance monitoring system.

5.9 Timer Task and RTOS Scheduling

The Timer Task and RTOS scheduling logic play an important role in the proposed FreeRTOS-Based Smart Attendance Monitoring System for managing periodic monitoring operations, telemetry generation, and efficient concurrent task execution under FreeRTOS architecture.

FreeRTOS utilizes a preemptive priority-based scheduling mechanism in which multiple tasks execute concurrently based on their assigned priorities and readiness conditions. The scheduler continuously monitors all tasks and allocates processor time according to task priority and system requirements.

The proposed system implements the following FreeRTOS tasks with different priorities:

Task	Priority
RFID Task	High
Processing Task	Medium
Firebase Task	Low
Timer Task	Lowest

The RFID Task is assigned the highest priority because RFID detection requires immediate response during attendance acquisition. The Processing Task executes after RFID detection and performs attendance verification and authorization checking. The Firebase Task handles cloud synchronization with lower priority because database communication can occur after attendance processing is completed.

The Timer Task is responsible for periodic monitoring and RTOS telemetry generation. It operates with the lowest priority because it mainly performs background monitoring operations and scheduling analysis.

The Timer Task continuously executes inside a periodic loop and generates RTOS execution traces for Serial Studio visualization. The task periodically transmits task scheduling information and execution timing through UART serial communication.

The telemetry generation logic implemented inside the Timer Task is shown below:

```
Serial.print("4,");  
Serial.println(millis());
```

In the telemetry output:

- 4 represents Timer Task identifier
- millis() represents current execution timing

The Timer Task utilizes FreeRTOS delay mechanisms for periodic execution. The delay operation implemented in the proposed system is shown below:

```
vTaskDelay(3000 / portTICK_PERIOD_MS);
```

In the above implementation:

- 3000 represents delay duration in milliseconds
- portTICK_PERIOD_MS converts delay into RTOS ticks

The FreeRTOS scheduler continuously switches between tasks depending on task availability and priority. Whenever a higher-priority task becomes ready for execution, the currently running lower-priority task is preempted automatically.

The RTOS scheduling operation implemented in the proposed system provides several advantages including:

- Efficient multitasking capability
- Priority-based task execution
- Improved processor utilization
- Faster real-time response
- Reliable concurrent execution
- Periodic monitoring support
- Efficient scheduling analysis
- Real-time telemetry generation

The Timer Task also plays an important role in RTOS scheduling visualization using Serial Studio. The telemetry traces generated by different tasks help in analyzing task execution order, scheduling behavior, and concurrent multitasking operation within the embedded system.

The implemented Timer Task and RTOS scheduling logic demonstrate practical utilization of FreeRTOS scheduling mechanisms, periodic task execution, and multitasking management within the proposed attendance monitoring system.

5.10 Serial Studio Real-Time Monitoring

Serial Studio real-time monitoring is implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for visualizing RTOS task execution behavior, scheduling flow, and multitasking activity during attendance operations. The visualization mechanism helps in analyzing FreeRTOS scheduling performance and understanding concurrent task execution within the embedded system.

The proposed system integrates Serial Studio with ESP32 through UART serial communication. FreeRTOS tasks generate telemetry traces during execution and transmit task execution information to Serial Studio for graphical visualization and scheduling analysis.

Each FreeRTOS task implemented in the system is assigned to a unique task identifier for telemetry monitoring purposes. The RFID Task, Processing Task, Firebase Task, and Timer Task continuously generate scheduling traces whenever they execute under the control of the FreeRTOS scheduler.

The telemetry information transmitted to Serial Studio mainly includes:

- Task identification number
- Task execution timing
- Scheduling sequence
- RTOS execution traces
- Periodic timer activity

The telemetry generation logic implemented in the proposed system is shown below:

```
Serial.print("1,");  
Serial.println(millis());
```

In the telemetry output:

- 1 represents RFID Task
- 2 represents Processing Task
- 3 represents Firebase Task
- 4 represents Timer Task
- millis() represents task execution timing

The UART telemetry data generated by ESP32 is received by Serial Studio and displayed graphically using real-time plots and scheduling graphs. The visualization graph shows task switching behavior, scheduling order, and concurrent execution flow of multiple FreeRTOS tasks.

During attendance operation, the RFID Task executes first whenever an RFID card is scanned. The Processing Task then validates attendance information, followed by Firebase synchronization and Timer Task execution. These scheduling transitions are visualized in real time through Serial Studio monitoring graphs.

The Serial Studio integration provides several advantages including:

- Real-time RTOS visualization
- Task scheduling analysis
- Concurrent execution monitoring
- Multitasking behavior observation
- Improved debugging capability
- Telemetry-based monitoring
- Processor activity analysis
- RTOS execution tracing

The real-time monitoring system also helps in verifying proper implementation of FreeRTOS concepts such as:

- Priority-based scheduling
- Concurrent task execution
- Queue communication
- Mutex synchronization
- Periodic task execution
- Scheduling preemption

The telemetry communication implemented in the proposed system introduces minimal processor overhead while providing efficient real-time scheduling analysis and visualization capability.

The Serial Studio real-time monitoring architecture demonstrates practical implementation of RTOS scheduling visualization and embedded multitasking analysis within the proposed FreeRTOS-Based Smart Attendance Monitoring System.

5.11 OLED Display Interface

The OLED display interface is implemented in the proposed FreeRTOS-Based Smart Attendance Monitoring System for providing real-time visual notifications, attendance status messages, and system interaction feedback during attendance operations.

The system utilizes a 128×64 SSD1306 OLED display module interfaced with the ESP32 microcontroller through the I2C communication protocol. The OLED display provides fast response time, low power consumption, compact size, and clear visibility suitable for embedded attendance monitoring applications.

The OLED display continuously shows real-time system information including:

- Attendance confirmation messages

- Student identification details
- Attendance session status
- Duplicate attendance warnings
- Unauthorized card alerts
- System initialization messages
- Attendance operation notifications

The OLED display is initialized during system startup using SSD1306 and Adafruit graphics libraries within the Arduino IDE development environment. The display interface configuration implemented in the proposed system is shown below:

```
Adafruit_SSD1306 display(
  SCREEN_WIDTH,
  SCREEN_HEIGHT,
  &Wire,
  -1
);
```

In the above implementation:

- SCREEN_WIDTH represents OLED width
- SCREEN_HEIGHT represents OLED height
- &Wire represents I2C communication interface

The OLED display initialization logic implemented during system startup is shown below:

```
display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
```

The display module communicates with ESP32 using the following I2C pin connections:

Task	Priority
RFID Task	High
Processing Task	Medium
Firebase Task	Low
Timer Task	Lowest

During attendance operations, the Processing Task updates the OLED display with attendance information and system notifications. When a valid RFID card is scanned, the student name and attendance confirmation message are displayed on the OLED screen.

The attendance display logic implemented in the proposed system is shown below:

```
display.clearDisplay();
```

```
display.setCursor(0, 10);  
display.println("Attendance Marked");
```

```
display.display();
```

Mutex synchronization mechanisms are implemented for safe OLED communication because multiple FreeRTOS tasks may attempt to access the display simultaneously. Before updating the display, the task acquires the display mutex to prevent concurrent access conflicts.

The OLED display interface provides several advantages including:

- Real-time visual notifications
- Fast display response
- Low power consumption
- Compact hardware integration
- Clear attendance status indication
- Improved user interaction
- Reliable embedded communication
- Efficient multitasking support

The OLED display continuously provides system feedback and improves operational monitoring capability during attendance management and RTOS multitasking execution.

The implemented OLED display interface demonstrates practical integration of I2C communication, FreeRTOS synchronization mechanisms, and embedded graphical display control within the proposed attendance monitoring system.

5.12 Firebase Database Implementation

The Firebase database implementation is an important part of the proposed FreeRTOS-Based Smart Attendance Monitoring System for cloud-based attendance storage, remote accessibility, and real-time attendance synchronization. Firebase Realtime Database is used as the cloud platform for storing attendance records generated during RFID attendance operations.

Firebase Realtime Database is a cloud-hosted NoSQL database provided by Google Firebase platform. It enables real-time data synchronization, remote database access, wireless communication, and efficient cloud storage suitable for IoT and embedded applications.

In the proposed system, Firebase database communication is handled by the Firebase Task implemented under FreeRTOS multitasking architecture. The ESP32 microcontroller establishes wireless communication with Firebase using integrated WiFi capability and Firebase authentication credentials.

The Firebase database stores the following attendance information:

- Student name
- RFID UID
- Attendance timing
- Attendance status
- Session information

During attendance operation, the Processing Task transfers verified attendance information to the Firebase Task through FreeRTOS queue communication. The Firebase Task formats the attendance data and uploads it to Firebase Realtime Database using Firebase APIs.

The Firebase configuration implemented in the proposed system is shown below:

```
config.host = FIREBASE_HOST;
```

```
config.signer.tokens.legacy_token = FIREBASE_AUTH;
```

```
Firebase.begin(&config, &auth);
```

In the above implementation:

- FIREBASE_HOST represents Firebase database URL
- FIREBASE_AUTH represents Firebase authentication key
- Firebase.begin() initializes Firebase communication

The attendance data upload logic implemented in the proposed system is shown below:

```
Firebase.setString(  
    fbdo,  
    path + "/UID",  
    fbData.uid  
);
```

The attendance timing information is uploaded using:

```
Firestore.setInt(
    fbdo,
    path + "/TIME",
    millis()
);
```

The attendance records are stored in structured database paths for organized cloud storage and easy attendance retrieval. The Firebase database architecture supports scalable attendance management and remote monitoring capability.

The Firebase database implementation provides several advantages including:

- Real-time attendance synchronization
- Cloud-based attendance storage
- Remote database accessibility
- Secure attendance management
- Efficient IoT integration
- Wireless communication support
- Improved system scalability
- Centralized attendance monitoring

The cloud-based architecture also enables future integration with:

- Mobile applications
- Web dashboards
- Attendance analytics
- Cloud reporting systems
- Remote institutional monitoring

The Firebase database implementation operates concurrently with other FreeRTOS tasks without affecting real-time attendance acquisition and multitasking execution.

The implemented Firebase database architecture demonstrates practical utilization of cloud-based IoT communication and FreeRTOS multitasking integration within the proposed attendance monitoring system.

ications.

Mutex synchronization mechanisms ensure safe OLED display access during concurrent task execution. FreeRTOS scheduling mechanisms manage execution order of all tasks based on task priority and readiness conditions.

The Timer Task periodically generates RTOS telemetry traces and scheduling information for Serial Studio visualization. UART communication transmits telemetry data to Serial Studio for graphical monitoring of multitasking behavior and task scheduling flow.

The overall software flow implemented in the proposed system provides several advantages including:

- Efficient multitasking execution
- Real-time attendance monitoring
- Reliable inter-task communication
- Safe resource synchronization
- Cloud-based attendance management
- RTOS scheduling visualization
- Concurrent task operation
- Improved processor utilization

The software flow architecture demonstrates practical implementation of FreeRTOS multitasking, RFID attendance acquisition, cloud synchronization, and RTOS visualization mechanisms within the proposed attendance monitoring system.

5.14 Summary

This chapter presented the software implementation details of the proposed FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, Arduino IDE, FreeRTOS multitasking architecture, Firebase cloud integration, and Serial Studio visualization mechanisms.

The software development environment, required libraries, and embedded programming architecture were explained in detail. Multiple FreeRTOS tasks were implemented for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring to achieve efficient concurrent task execution and real-time attendance management.

The chapter also discussed queue-based inter-task communication mechanisms used for reliable data transfer between concurrent tasks and mutex synchronization techniques implemented for safe access to shared hardware resources such as the OLED display module.

The implementation of RTOS scheduling mechanisms, software timers, and telemetry generation for Serial Studio visualization was presented. Real-time task execution traces generated through UART communication enabled graphical scheduling analysis and multitasking visualization.

Firestore cloud integration and database implementation were also explained in detail. Attendance information including student name, RFID UID, and timing information was successfully uploaded to Firestore Realtime Database through WiFi communication for cloud-based attendance monitoring and remote accessibility.

The OLED display interface, attendance processing logic, duplicate attendance detection, attendance session control, and unauthorized RFID card handling mechanisms were also implemented successfully within the software architecture.

The overall software implementation demonstrated practical utilization of FreeRTOS concepts such as:

- Multitasking
- Priority-based scheduling
- Queue communication
- Mutex synchronization
- Software timers
- Concurrent task execution
- RTOS visualization

The implemented software architecture achieved reliable attendance monitoring, efficient processor utilization, real-time cloud synchronization, stable multitasking operation, and successful integration of embedded RTOS concepts.

The next chapter presents the experimental results, system outputs, RTOS scheduling analysis, performance evaluation, and overall testing observations obtained from the proposed attendance monitoring system.

CHAPTER 6 RESULTS AND ANALYSIS

6.1 Introduction

This chapter presents the experimental results, system outputs, performance evaluation, and RTOS scheduling analysis of the proposed FreeRTOS-Based Smart Attendance Monitoring System implemented using ESP32 microcontroller, RC522 RFID reader, Firestore cloud integration, and FreeRTOS multitasking architecture.

The experimental analysis was carried out to verify the functionality, reliability, multitasking capability, cloud synchronization performance, and RTOS scheduling behavior of the proposed system under different attendance monitoring conditions.

The implemented system was tested using multiple RFID cards for attendance acquisition, session control, duplicate attendance verification, unauthorized card detection, and cloud synchronization operations. FreeRTOS task execution, queue communication, mutex synchronization, software timer operation, and multitasking scheduling behavior were analyzed using Serial Studio real-time visualization.

The experimental setup included ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, WiFi communication, Firebase Realtime Database, and Serial Studio telemetry monitoring interface. Attendance records were uploaded to Firebase cloud database during real-time attendance operation.

The results obtained from the experimental analysis demonstrate successful implementation of:

- RFID-based attendance monitoring
- FreeRTOS multitasking execution
- Queue-based inter-task communication
- Mutex synchronization
- Firebase cloud synchronization
- RTOS scheduling visualization
- Duplicate attendance prevention
- Unauthorized RFID card detection
- Real-time telemetry monitoring

This chapter explains the detailed experimental observations, system outputs, scheduling analysis, telemetry visualization results, and performance evaluation of the proposed attendance monitoring system.

6.2 RFID Detection Results

The RFID detection results obtained from the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrate reliable RFID card acquisition, real-time attendance detection, and stable operation of the RC522 RFID reader under FreeRTOS multitasking architecture.

The RC522 RFID reader was tested using multiple RFID cards assigned to different students and attendance control operations. The RFID Task continuously monitored RFID card presence and successfully detected RFID UIDs whenever RFID cards were placed near the reader module.

During experimental testing, the RFID reader successfully identified both master RFID cards and student RFID cards with fast response time and stable communication. The RFID detection process operated continuously under the FreeRTOS scheduler without interrupting other concurrent tasks such as Firebase communication and telemetry generation.

The experimental results showed that the RFID reader successfully performed the following operations:

- RFID card detection
- RFID UID extraction
- Student card identification
- Master card identification
- Real-time attendance acquisition
- Unauthorized card detection

When a valid RFID card was scanned, the RFID Task extracted the RFID UID and transferred the information to the Processing Task using FreeRTOS queue communication mechanisms. The queue transfer operation was completed successfully without data corruption or synchronization issues.

The RFID reader also demonstrated stable SPI communication with the ESP32 microcontroller throughout the attendance monitoring process. No major communication errors or RFID reading failures were observed during experimental operation.

The OLED display successfully showed attendance confirmation messages after RFID card detection, while buzzer and LED indicators generated immediate audio-visual feedback for attendance operations.

The experimental observations confirmed that the RFID reader provided:

- Fast RFID detection response
- Stable RFID communication
- Reliable UID acquisition
- Contactless attendance operation
- Efficient multitasking compatibility
- Real-time attendance monitoring

The RFID Task also generated telemetry traces successfully for Serial Studio visualization during attendance operation. Real-time task execution traces confirmed proper execution of the RFID Task under FreeRTOS scheduling architecture.

The successful RFID detection results demonstrate efficient integration of RFID technology, SPI communication, FreeRTOS multitasking, and embedded attendance monitoring within the proposed system.

6.3 Attendance Monitoring Results

The attendance monitoring results obtained from the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrate successful implementation of real-time RFID-based attendance acquisition, attendance verification, duplicate attendance prevention, and attendance session management using FreeRTOS multitasking architecture.

The attendance monitoring process was experimentally tested using multiple student RFID cards and a master RFID card for attendance session control. The system successfully detected valid RFID cards and processed attendance information in real time without significant response delay.

During attendance operation, the master RFID card was used for activating and terminating attendance sessions. The Processing Task correctly identified the master card and controlled the attendance session state successfully.

When valid student RFID cards were scanned during active attendance sessions, the system successfully performed the following operations:

- Student identification
- Attendance verification
- Attendance confirmation
- Attendance storage
- OLED display notification
- Audio-visual indication
- Cloud synchronization

The Processing Task correctly matched scanned RFID UIDs with registered student records and displayed student attendance confirmation messages through the OLED display. The green LED and buzzer generated successful attendance indications immediately after attendance verification.

The duplicate attendance prevention mechanism also operated successfully during experimental testing. When the same RFID card was scanned multiple times within the same attendance session, the system identified duplicate attendance entries and generated duplicate warning notifications.

The duplicate attendance detection system successfully performed the following functions:

- Duplicate attendance identification
- Duplicate warning generation
- OLED warning notification

- Buzzer warning indication
- Red LED activation

Unauthorized RFID card detection was also verified experimentally using unregistered RFID cards. The system correctly identified invalid RFID cards and generated unauthorized access warning messages through the OLED display and buzzer indications.

The attendance monitoring operation demonstrated stable multitasking performance under FreeRTOS scheduling architecture. RFID detection, attendance processing, Firebase communication, OLED updates, and telemetry generation executed concurrently without affecting system stability.

The attendance monitoring results confirmed that the proposed system provided:

- Real-time attendance acquisition
- Reliable attendance verification
- Fast attendance response
- Duplicate attendance prevention
- Unauthorized card detection
- Stable multitasking operation
- Efficient user notification
- Reliable attendance management

The successful attendance monitoring results demonstrate effective integration of RFID technology, FreeRTOS multitasking, queue communication, synchronization mechanisms, and embedded attendance management within the proposed system.

6.4 Firebase Cloud Upload Results

The Firebase cloud upload results obtained from the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrate successful real-time attendance synchronization and reliable cloud-based attendance storage using ESP32 WiFi communication and Firebase Realtime Database.

The Firebase communication system was experimentally tested during multiple attendance operations using valid RFID cards. After successful attendance verification by the Processing Task, attendance information was transferred to the Firebase Task through FreeRTOS queue communication mechanisms for cloud synchronization.

The Firebase Task successfully established WiFi communication between the ESP32 microcontroller and Firebase Realtime Database using predefined network credentials and

Firebase authentication parameters. Stable wireless communication was maintained throughout the attendance monitoring operation.

During experimental testing, the following attendance information was successfully uploaded to Firebase cloud database:

- Student name
- RFID UID
- Attendance timing
- Attendance status

The attendance upload operation was performed automatically whenever valid student attendance was detected. Attendance records were stored in structured Firebase database paths for easy cloud management and remote attendance monitoring.

The Firebase upload process operated concurrently with other FreeRTOS tasks such as RFID scanning and telemetry generation without interrupting real-time attendance acquisition. Queue communication mechanisms ensured reliable transfer of attendance information between the Processing Task and Firebase Task.

The experimental observations confirmed that the Firebase cloud synchronization system provided:

- Real-time attendance upload
- Stable WiFi communication
- Reliable cloud synchronization
- Remote attendance accessibility
- Concurrent multitasking operation
- Automatic attendance storage
- Cloud-based attendance monitoring

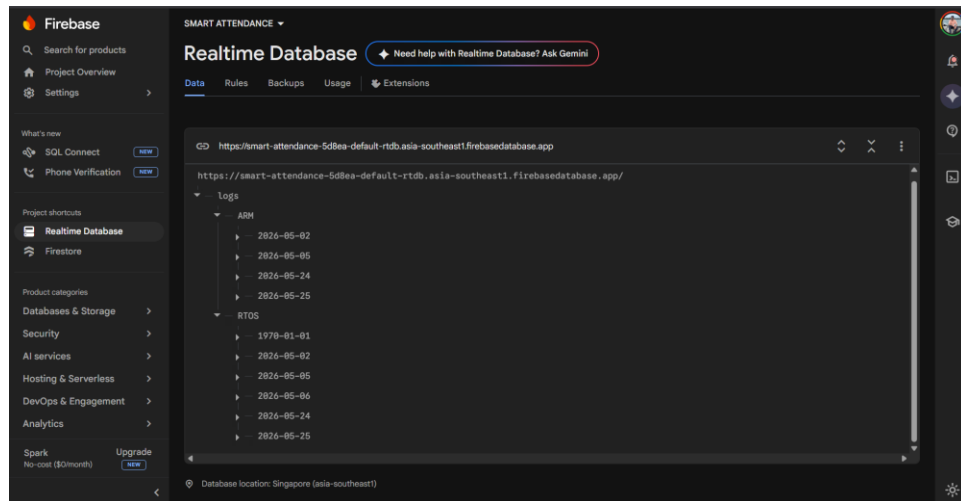


Figure 6.1: Firebase Realtime Database Showing Attendance Records

The Firebase upload response time was observed to be stable under normal WiFi network conditions. Attendance information was synchronized successfully without major communication failures or data corruption during experimental operation.

The OLED display and serial monitor also confirmed successful cloud synchronization by displaying Firebase upload status messages during attendance operations.

The Firebase Task telemetry traces generated during attendance upload operations were visualized successfully in Serial Studio, confirming proper concurrent execution of cloud synchronization operations under FreeRTOS scheduling architecture.

The experimental results demonstrate successful implementation of IoT-based cloud communication, wireless attendance synchronization, and FreeRTOS multitasking integration within the proposed attendance monitoring system.

6.5 FreeRTOS Task Scheduling Analysis

The FreeRTOS task scheduling analysis carried out on the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrated efficient multitasking operation, concurrent task execution, and proper implementation of priority-based scheduling mechanisms using FreeRTOS on ESP32 microcontroller.

The proposed system implemented multiple FreeRTOS tasks including RFID Task, Processing Task, Firebase Task, and Timer Task for handling attendance acquisition, attendance verification, cloud synchronization, and telemetry monitoring concurrently.

The FreeRTOS scheduler continuously managed execution of all tasks based on assigned task priorities and task readiness conditions. The experimental observations confirmed that higher-priority tasks were executed before lower-priority tasks whenever required, ensuring efficient real-time responsiveness during attendance operations.

The RFID Task was assigned the highest priority because RFID card detection required immediate processing during attendance acquisition. Whenever an RFID card was scanned, the RFID Task immediately became active and processed RFID information successfully without noticeable delay.

The Processing Task operated after RFID data reception and performed attendance verification, duplicate detection, and authorization checking. The Firebase Task executed independently for cloud synchronization operations while the Timer Task periodically generated telemetry traces for scheduling visualization.

The scheduling sequence observed during attendance operation was as follows:

RFID Task → Processing Task → Firebase Task → Timer Task

The FreeRTOS scheduler successfully handled task preemption and concurrent execution during experimental testing. Higher-priority tasks interrupted lower-priority tasks whenever immediate processing was required, demonstrating correct implementation of preemptive scheduling mechanisms.

The task scheduling behavior was visualized using Serial Studio through UART telemetry communication. Task execution traces generated by FreeRTOS tasks were displayed graphically in real time, enabling observation of multitasking behavior and scheduling flow.

The scheduling analysis confirmed the following RTOS functionalities:

- Priority-based scheduling
- Concurrent task execution
- Task preemption
- Real-time task switching
- Efficient processor utilization
- Queue-based task synchronization
- Periodic task execution
- Stable multitasking operation

The experimental observations also showed that FreeRTOS delay mechanisms and blocked task states improved processor utilization by preventing unnecessary CPU activity during idle periods.

The Timer Task periodically generated telemetry traces at predefined intervals without affecting attendance acquisition and cloud synchronization operations. Queue communication mechanisms enabled safe data transfer between tasks while maintaining stable multitasking execution.

The task scheduling analysis demonstrated several advantages of the implemented FreeRTOS architecture including:

- Faster real-time response
- Efficient concurrent operation
- Improved processor utilization
- Stable multitasking performance
- Reliable scheduling behavior
- Reduced response delay
- Better task coordination
- Practical RTOS implementation

The successful scheduling analysis confirms effective implementation of FreeRTOS multitasking, task prioritization, queue communication, and scheduling mechanisms within the proposed attendance monitoring system.

6.6 Queue Communication Analysis

The queue communication analysis performed on the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrated reliable inter-task communication, safe data transfer, and efficient synchronization between concurrent FreeRTOS tasks.

The proposed system implemented queue-based communication mechanisms for transferring RFID UID information and attendance data between the RFID Task, Processing Task, and Firebase Task. The FreeRTOS queues operated successfully during all attendance monitoring operations without data corruption or communication failure.

Two major queues were implemented in the proposed system:

- RFID Queue
- Firebase Queue

The RFID Queue was used for transferring RFID UID information from the RFID Task to the Processing Task. During experimental testing, whenever an RFID card was scanned, the RFID Task successfully extracted the RFID UID and transmitted the data to the Processing Task using FreeRTOS queue APIs.

The Processing Task continuously waited for RFID information from the queue and successfully received attendance data without communication delay or synchronization issues. The queue communication mechanism ensured reliable transfer of RFID information even during concurrent execution of multiple FreeRTOS tasks.

After attendance verification, attendance information such as student name, RFID UID, and attendance timing was transmitted to the Firebase Task using the Firebase Queue. The Firebase Task successfully retrieved attendance data from the queue and uploaded the information to Firebase Realtime Database for cloud synchronization.

The queue communication architecture demonstrated stable operation during:

- RFID attendance acquisition
- Attendance verification
- Cloud synchronization
- Concurrent multitasking execution
- RTOS scheduling visualization

The experimental observations confirmed that queue communication mechanisms provided:

- Reliable inter-task communication
- Safe data transfer
- Efficient multitasking coordination
- Reduced synchronization complexity
- Organized task interaction
- Improved processor utilization
- Stable concurrent execution
- Reliable attendance data handling

The queue communication system also improved processor efficiency because tasks waiting for queue data remained in blocked state until new information became available. This prevented unnecessary CPU activity and improved overall multitasking performance.

No major queue overflow conditions, synchronization conflicts, or communication failures were observed during experimental operation. The queue communication architecture maintained stable performance throughout attendance monitoring and cloud synchronization processes.

The FreeRTOS queue APIs successfully handled producer-consumer communication between concurrent tasks while maintaining reliable data integrity and scheduling stability.

The queue communication analysis demonstrates effective implementation of FreeRTOS inter-task communication mechanisms for real-time embedded attendance monitoring and concurrent multitasking operation within the proposed system.

6.7 Mutex Synchronization Analysis

The mutex synchronization analysis performed on the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrated reliable protection of shared resources, stable concurrent task execution, and efficient synchronization management under FreeRTOS multitasking architecture.

The proposed system implemented mutex synchronization mechanisms mainly for protecting OLED display communication during concurrent task execution. Since multiple FreeRTOS tasks could attempt to access the OLED display simultaneously, mutex synchronization was necessary to prevent resource conflicts and unstable display operation.

During experimental testing, the OLED display was continuously updated with attendance confirmation messages, duplicate attendance warnings, session status notifications, and unauthorized card alerts. Multiple tasks attempted to access the display resource during attendance operation; however, mutex synchronization successfully ensured exclusive access to the OLED display at any given time.

The FreeRTOS mutex APIs operated correctly during all multitasking operations. Before updating the OLED display, the task successfully acquired the display mutex and released it after completing the display operation.

The experimental observations confirmed that mutex synchronization successfully prevented:

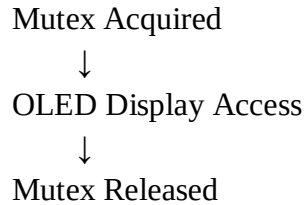
- Simultaneous display access conflicts
- Overlapping OLED messages
- Display communication corruption
- Resource synchronization issues
- Unstable multitasking behavior

The mutex synchronization mechanism also improved system stability during concurrent operation of RFID scanning, attendance processing, Firebase communication, and telemetry generation tasks.

The mutex synchronization process followed the sequence below:

Task Requests Mutex





The experimental analysis confirmed that the implemented mutex synchronization architecture provided:

- Safe shared resource access
- Reliable OLED communication
- Improved multitasking stability
- Protected concurrent execution
- Efficient resource management
- Reduced synchronization conflicts
- Better task coordination
- Stable RTOS operation

No major synchronization failures, display corruption issues, or concurrent access conflicts were observed during experimental attendance monitoring operations.

The FreeRTOS mutex mechanism also supported priority inheritance during multitasking execution, reducing the possibility of priority inversion problems during shared resource access.

The mutex synchronization analysis demonstrates successful implementation of FreeRTOS synchronization mechanisms for reliable concurrent task execution and safe shared resource management within the proposed attendance monitoring system.

6.8 Serial Studio Graph Analysis

The Serial Studio graph analysis performed on the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrated successful real-time visualization of RTOS task execution, scheduling behavior, and concurrent multitasking operation using UART telemetry communication.

The proposed system integrated Serial Studio with the ESP32 microcontroller for monitoring FreeRTOS task execution and scheduling flow graphically. Each FreeRTOS task generated telemetry traces containing task identifiers and execution timing information during runtime.

The following task identifiers were used for telemetry visualization:

Task Identifier	FreeRTOS Task
1	RFID Task
2	Processing Task
3	Firebase Task
4	Timer Task

During attendance operation, telemetry data generated by FreeRTOS tasks was transmitted to Serial Studio through UART serial communication. Serial Studio received the telemetry information and displayed real-time scheduling graphs representing task execution flow and multitasking behavior.

The experimental visualization graphs confirmed successful execution of:

- RFID Task scheduling
- Attendance Processing Task execution
- Firebase synchronization activity
- Timer Task periodic execution
- Concurrent multitasking operation
- Real-time scheduling transitions

Whenever an RFID card was scanned, the Serial Studio graph showed immediate activation of the RFID Task followed by Processing Task execution. After attendance verification, the Firebase Task became active for cloud synchronization while the Timer Task periodically generated monitoring traces.

The observed scheduling sequence during attendance operation was:

RFID Task → Processing Task → Firebase Task → Timer Task

The Serial Studio visualization graphs successfully demonstrated:

- Task switching behavior
- Priority-based scheduling
- Concurrent task execution
- Real-time multitasking activity
- Periodic timer operation
- RTOS scheduling flow
- UART telemetry monitoring
- Processor activity visualization

The telemetry traces generated by FreeRTOS tasks were stable throughout experimental operation. No major telemetry transmission failures or visualization interruptions were observed during attendance monitoring.

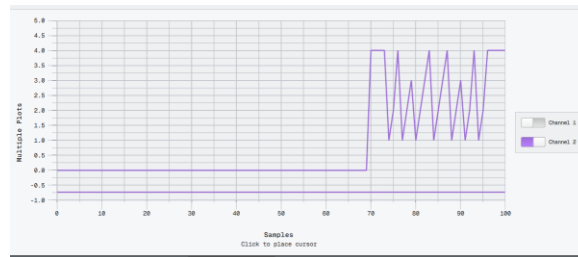


Figure 6.2: Real-Time FreeRTOS Task Scheduling Graph using Serial Studio

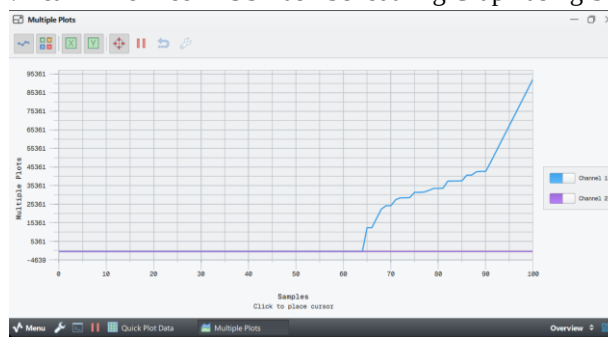


Figure 6.3: Concurrent Task Execution Visualization

The graphical scheduling visualization also confirmed correct implementation of:

- Preemptive scheduling
- Queue communication
- Concurrent task management
- Periodic task execution
- RTOS multitasking architecture

The experimental observations confirmed that the Serial Studio integration provided several advantages including:

- Real-time RTOS visualization
- Easy scheduling analysis
- Improved debugging capability
- Practical multitasking demonstration
- Telemetry-based monitoring
- Processor activity observation

- Efficient task tracing
- Educational RTOS analysis support

6.9 Hardware Output Analysis

The hardware output analysis performed on the proposed FreeRTOS-Based Smart Attendance Monitoring System demonstrated successful operation of all hardware peripherals including the ESP32 microcontroller, RC522 RFID reader, OLED display module, buzzer, LED indicators, WiFi communication system, and Serial Studio telemetry interface.

The complete hardware setup operated reliably during attendance monitoring operations under FreeRTOS multitasking architecture. All hardware modules communicated successfully with the ESP32 microcontroller through SPI, I2C, GPIO, UART, and WiFi communication protocols.

The RC522 RFID reader successfully detected RFID cards and transmitted RFID UID information to the ESP32 without communication failures. RFID scanning response time was observed to be fast and stable during multiple attendance operations.

The OLED display module continuously displayed attendance notifications and system status information during attendance monitoring. The display successfully showed:

- Attendance confirmation messages
- Student identification details
- Session activation status
- Duplicate attendance warnings
- Unauthorized card alerts
- System initialization messages

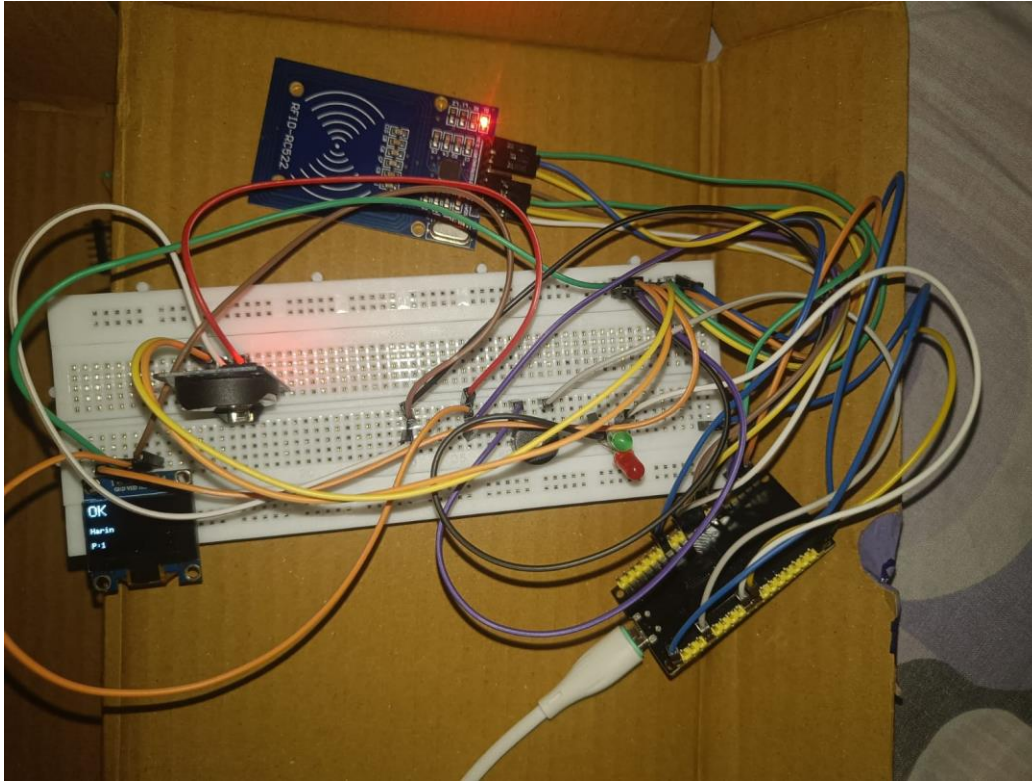


Fig. 6.4 Output when student tapped the card to enter the class

The buzzer and LED indicators also operated successfully during attendance processing. The green LED and buzzer generated successful attendance indications for valid RFID cards, while the red LED and warning buzzer were activated during duplicate attendance detection and invalid RFID card access.

The ESP32 integrated WiFi module maintained stable wireless communication with Firebase Realtime Database during cloud synchronization operations. Attendance information was uploaded successfully to the cloud database during experimental testing.

The Serial Studio interface successfully visualized RTOS task execution and scheduling behavior through UART telemetry communication. Real-time task execution traces generated by FreeRTOS tasks were displayed graphically without significant communication delay.

The hardware output analysis confirmed successful operation of the following system functionalities:

- RFID-based attendance acquisition
- OLED display communication

- Audio-visual attendance indication
- Cloud synchronization
- UART telemetry generation
- RTOS scheduling visualization
- FreeRTOS multitasking execution
- Queue communication operation
- Mutex synchronization handling

The experimental observations confirmed that the hardware architecture provided:

- Stable embedded operation
- Reliable hardware communication
- Efficient multitasking performance
- Real-time attendance monitoring
- Fast response capability
- Reliable cloud synchronization
- Efficient telemetry transmission
- Stable peripheral interfacing

No major hardware communication failures, power instability issues, or peripheral synchronization problems were observed during system operation. The hardware components operated continuously and reliably under concurrent multitasking conditions.

The hardware output analysis demonstrates successful integration of embedded hardware peripherals, FreeRTOS multitasking mechanisms, cloud communication architecture, and RTOS scheduling visualization within the proposed attendance monitoring system.

6.10 Performance Evaluation

The performance evaluation of the proposed FreeRTOS-Based Smart Attendance Monitoring System was carried out to analyze system efficiency, multitasking capability, response time, scheduling performance, cloud synchronization reliability, and overall embedded system stability under real-time attendance monitoring conditions.

The proposed system was experimentally tested using multiple RFID cards, concurrent FreeRTOS tasks, Firebase cloud communication, and Serial Studio telemetry monitoring. The evaluation focused on analyzing the operational behavior of the system during continuous attendance acquisition and multitasking execution.

The RFID attendance acquisition process demonstrated fast response time and reliable RFID detection performance. The RFID reader successfully detected RFID cards within a short

duration after card placement near the RC522 reader module. Attendance verification and display notification operations were completed efficiently without noticeable response delay.

The FreeRTOS multitasking architecture successfully handled concurrent execution of RFID scanning, attendance processing, Firebase communication, and telemetry generation tasks. The FreeRTOS scheduler efficiently managed task switching and priority-based execution throughout the attendance monitoring operation.

The queue communication mechanisms demonstrated reliable inter-task data transfer between FreeRTOS tasks without communication failure or data corruption. Mutex synchronization mechanisms also ensured safe OLED display communication during concurrent task execution.

The Firebase cloud synchronization system maintained stable WiFi communication and successfully uploaded attendance records to Firebase Realtime Database during attendance operation. Cloud synchronization performance remained stable under normal wireless network conditions.

The Serial Studio telemetry visualization system successfully generated real-time scheduling graphs and multitasking execution traces without affecting overall system performance. UART telemetry communication introduced minimal processor overhead during scheduling analysis.

The experimental observations confirmed the following performance characteristics of the proposed system:

Performance Parameter	Observation
RFID Detection	Fast and Stable
Attendance Verification	Reliable
FreeRTOS Scheduling	Efficient
Queue Communication	Stable
Mutex Synchronization	Successful
Firebase Upload	Reliable
UART Telemetry	Stable
Multitasking Performance	Efficient
OLED Response	Fast
System Stability	High

The proposed system demonstrated several performance advantages including:

- Efficient multitasking capability
- Fast attendance response
- Reliable concurrent execution
- Stable cloud synchronization

- Efficient processor utilization
- Real-time scheduling analysis
- Reliable inter-task communication
- Stable embedded system operation

The experimental evaluation confirmed that the proposed FreeRTOS-based architecture significantly improved multitasking efficiency and scheduling capability compared to conventional sequential attendance monitoring systems.

The performance evaluation results demonstrate successful implementation of FreeRTOS multitasking, RFID attendance acquisition, cloud communication, synchronization mechanisms, and RTOS scheduling visualization within the proposed attendance monitoring system.

6.11 Summary

This chapter presented the experimental results, system outputs, RTOS scheduling analysis, and performance evaluation of the proposed FreeRTOS-Based Smart Attendance Monitoring System implemented using ESP32 microcontroller, RC522 RFID reader, Firebase cloud integration, and FreeRTOS multitasking architecture.

The experimental analysis confirmed successful implementation of RFID-based attendance acquisition, attendance verification, duplicate attendance prevention, unauthorized RFID card detection, cloud synchronization, and real-time multitasking operation. The RC522 RFID reader successfully detected RFID cards and transferred RFID UID information reliably to the ESP32 microcontroller for attendance processing.

The attendance monitoring system operated efficiently under FreeRTOS scheduling architecture using multiple concurrent tasks for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring. Queue communication mechanisms successfully enabled reliable inter-task data transfer, while mutex synchronization ensured safe access to shared resources such as the OLED display module.

Firebase cloud synchronization was successfully achieved through WiFi communication, enabling real-time attendance upload and remote attendance monitoring using Firebase Realtime Database. The Serial Studio visualization system successfully displayed RTOS scheduling behavior and task execution flow using UART telemetry communication.

The hardware output analysis demonstrated stable operation of all embedded peripherals including OLED display notifications, buzzer indications, LED status monitoring, cloud synchronization, and telemetry visualization. The FreeRTOS scheduler efficiently managed concurrent task execution and priority-based scheduling during attendance operations.

The overall performance evaluation confirmed that the proposed system provided:

- Reliable RFID attendance monitoring
- Efficient multitasking capability
- Stable RTOS scheduling operation
- Reliable queue communication
- Safe mutex synchronization
- Real-time cloud synchronization
- Efficient processor utilization
- Practical RTOS visualization support

The experimental results demonstrated successful integration of RFID technology, FreeRTOS multitasking, cloud communication, synchronization mechanisms, and embedded scheduling visualization within a unified real-time attendance monitoring architecture.

The next chapter presents the conclusion, advantages, future scope, and overall contributions of the proposed FreeRTOS-Based Smart Attendance Monitoring System.

CHAPTER 7 CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

The proposed FreeRTOS-Based Smart Attendance Monitoring System was successfully designed and implemented using ESP32 microcontroller, RC522 RFID reader, Firebase cloud integration, OLED display module, and FreeRTOS multitasking architecture. The system demonstrated reliable real-time attendance acquisition, efficient concurrent task execution, cloud synchronization, and RTOS scheduling visualization suitable for embedded and IoT-based applications.

The implemented system successfully utilized FreeRTOS concepts such as task scheduling, queue-based inter-task communication, mutex synchronization, software timers, and concurrent task execution for improving multitasking efficiency and real-time responsiveness. Separate FreeRTOS tasks were implemented for RFID scanning, attendance processing, Firebase communication, and timer-based monitoring operations.

The RC522 RFID reader successfully performed contactless attendance acquisition and reliable RFID UID detection during attendance monitoring operations. Attendance session control, duplicate attendance prevention, and unauthorized RFID card detection mechanisms operated successfully under the FreeRTOS multitasking environment.

Firestore Realtime Database integration enabled real-time attendance synchronization and cloud-based attendance management through WiFi communication. Attendance information including student name, RFID UID, and attendance timing was uploaded successfully to the cloud database for remote accessibility and centralized attendance storage.

Queue communication mechanisms ensured reliable data transfer between concurrent FreeRTOS tasks, while mutex synchronization protected shared resources such as OLED display communication from simultaneous access conflicts. The FreeRTOS scheduler efficiently managed priority-based task execution and concurrent multitasking operations throughout the system execution.

The integration of Serial Studio enabled successful visualization of RTOS scheduling behavior and multitasking execution flow through UART telemetry communication. Real-time scheduling graphs generated by the system demonstrated practical implementation of RTOS concepts such as task switching, preemptive scheduling, and concurrent task execution.

The experimental results confirmed that the proposed system provided:

- Reliable RFID attendance monitoring
- Efficient multitasking capability
- Stable RTOS scheduling operation
- Real-time cloud synchronization
- Reliable queue communication
- Safe mutex synchronization
- Efficient processor utilization
- Practical RTOS visualization support

The proposed system successfully demonstrated practical integration of RFID technology, FreeRTOS multitasking, cloud communication, synchronization mechanisms, and embedded scheduling visualization within a unified real-time attendance monitoring architecture suitable for educational and embedded system applications.

7.2 Future Scope

The proposed FreeRTOS-Based Smart Attendance Monitoring System can be further enhanced and expanded in several ways to improve functionality, scalability, security, and real-time embedded performance. The current system successfully demonstrates RFID-based attendance monitoring, FreeRTOS multitasking, cloud synchronization, and RTOS scheduling visualization; however, additional advanced features can be integrated in future developments.

One important future enhancement is the integration of biometric authentication techniques such as fingerprint recognition and facial recognition along with RFID technology for improving attendance security and preventing proxy attendance more effectively.

The system can also be expanded using mobile applications and web-based dashboards for remote attendance monitoring, attendance analytics, and cloud-based institutional management. Android applications can be integrated with Firebase for real-time attendance notifications and student attendance tracking.

Future improvements may also include:

- GPS-based location verification
- Face recognition attendance systems
- AI-based attendance analytics
- Cloud-based report generation
- Mobile attendance applications
- Web dashboard integration
- Multi-classroom attendance management
- Student database management system
- SMS and email attendance alerts
- QR code-based attendance monitoring
- Voice notification systems
- Advanced IoT analytics integration

The proposed system can also be extended for large-scale institutional deployment using centralized cloud servers and distributed IoT architectures for handling multiple classrooms and departments simultaneously.

Additional RTOS features such as dynamic task management, CPU load monitoring, advanced scheduling algorithms, and power optimization techniques can also be implemented for improving real-time system performance and embedded resource utilization.

Future versions of the system may integrate advanced communication technologies such as MQTT, LoRa, ZigBee, or BLE for long-range wireless communication and large-scale IoT deployment.

The Serial Studio visualization system can further be extended with advanced RTOS performance analysis tools, processor load monitoring, task execution statistics, and real-time debugging frameworks for embedded research applications.

The proposed architecture also provides a strong foundation for developing smart campus automation systems, industrial monitoring applications, and advanced IoT-enabled embedded monitoring platforms using FreeRTOS and ESP32.

7.3 Summary

This chapter presented the conclusion, advantages, and future scope of the proposed FreeRTOS-Based Smart Attendance Monitoring System developed using ESP32 microcontroller, RC522 RFID reader, Firebase cloud integration, and FreeRTOS multitasking architecture.

The proposed system successfully demonstrated real-time RFID-based attendance acquisition, concurrent task execution, queue-based inter-task communication, mutex synchronization, cloud synchronization, and RTOS scheduling visualization using embedded FreeRTOS mechanisms.

The implementation of multiple FreeRTOS tasks for RFID scanning, attendance processing, Firebase communication, and timer monitoring significantly improved multitasking capability, processor utilization, scheduling efficiency, and real-time responsiveness of the attendance monitoring system.

The integration of Firebase Realtime Database enabled real-time attendance synchronization and remote attendance monitoring through WiFi communication. The Serial Studio visualization system successfully demonstrated RTOS scheduling behavior and concurrent task execution using UART telemetry communication.

The advantages of the proposed system over conventional attendance systems were also discussed. The implemented architecture provided efficient multitasking capability, reliable attendance management, cloud-based storage, scheduling visualization, and practical implementation of embedded RTOS concepts suitable for educational and research applications.

The future scope of the project includes integration of biometric authentication, mobile applications, advanced IoT communication technologies, AI-based attendance analytics, web dashboard systems, and large-scale institutional monitoring architectures.

Overall, the proposed FreeRTOS-Based Smart Attendance Monitoring System successfully demonstrated practical implementation of RFID technology, embedded multitasking, cloud communication, synchronization mechanisms, and RTOS scheduling visualization within a unified real-time embedded attendance monitoring architecture.

CHAPTER 8 REFERENCES

1. Richard Barry, *Mastering the FreeRTOS Real Time Kernel*, Real Time Engineers Ltd., United Kingdom, 2021.
2. FreeRTOS Documentation, “FreeRTOS Kernel Developer Documentation,” Available: [FreeRTOS Official Website](#)
3. Espressif Systems, “ESP32 Technical Reference Manual,” Available: [ESP32 Official Documentation](#)
4. Arduino Documentation, “Arduino IDE and ESP32 Programming Guide,” Available: [Arduino Official Website](#)
5. NXP Semiconductors, “MFRC522 RFID Reader Datasheet,” Available: [NXP Semiconductors](#)
6. Firebase Documentation, “Firebase Realtime Database Documentation,” Available: [Firebase Official Website](#)
7. Adafruit Industries, “SSD1306 OLED Display Library Documentation,” Available: [Adafruit Official Website](#)
8. J. Liu, *Real-Time Systems*, Pearson Education, 2017.
9. Qing Li and Caroline Yao, *Real-Time Concepts for Embedded Systems*, CMP Books, 2015.
10. M. Margolis, *Arduino Cookbook*, O’Reilly Media, 2020.
11. Raj Kamal, *Embedded Systems: Architecture, Programming and Design*, McGraw Hill Education, 2018.
12. Simon Monk, *Programming Arduino: Getting Started with Sketches*, McGraw Hill, 2019.
13. K. V. K. K. Prasad, “RFID Based Smart Attendance Monitoring System Using IoT,” *International Journal of Engineering Research and Technology (IJERT)*, vol. 9, no. 5, pp. 112–118, 2020.
14. S. Sharma and A. Verma, “Real-Time Attendance System Using RFID and ESP32,” *International Journal of Advanced Research in Computer Engineering and Technology*, vol. 8, no. 3, pp. 45–51, 2021.
15. P. Gupta and R. Singh, “Implementation of FreeRTOS for Embedded Monitoring Applications,” *International Journal of Embedded Systems and Applications*, vol. 11, no. 2, pp. 23–31, 2022.
16. Serial Studio Documentation, “Serial Studio User Guide and Telemetry Visualization,” Available: [Serial Studio Official Website](#)